Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications

Rapport de projet transversal (PPP)
Configuration, intégration, validation et démonstration

École Centrale des Logiciels Libres et de Télécommunications (EC2LT)

Groupe 3

Housseinou Yakham DIAGNE — Rui ATA JUNIOR — Samsoudine TOMBOANJARA WILDINE

Dieynaba BA — Mohamed Ali M'djara RACHAD — Clevy Floden BANZOULOU — El Hadji GUEYE

Année 2026 — Laboratoire Containerlab « mcp-noc-lab »

Sommaire

PARTIE I — Cadre théorique et étude documentaire

1. La supervision dans un centre opérationnel (NOC)
2. Le Model Context Protocol (MCP)
3. Les protocoles de supervision : SNMP, NETCONF, CLI, NetFlow, PCAP
4. Le routage de redondance : OSPF et iBGP
5. Technologies retenues et justification
6. Analyse du cahier des charges et grille de conformité

PARTIE II — Mise en œuvre

7. Architecture de la solution
8. Mise en œuvre par scénarios (S0 → S9)
9. Sécurité mise en œuvre et gestion des secrets

PARTIE III — Méthodologie de tests (solidité, robustesse, sécurité)

10. Stratégie générale d'essais
11. Niveau 1 — Tests unitaires
12. Niveau 2 — Tests d'intégration contre le laboratoire réel
13. Niveau 3 — Tests de bout en bout (protocole MCP)
14. Niveau 4 — Tests de robustesse
15. Niveau 5 — Tests de sécurité
16. Grille de résultats et critères d'acceptation
17. Protocole de campagne de tests (« comment tester de façon générale »)

PARTIE IV — Démonstrations pour un public non informaticien

18. Préparer le public : analogies et vocabulaire simple
19. Les scénarios de démonstration (A à G)
20. Déroulé type de la soutenance (B.9)

PARTIE V — Dimensionnement, calculs et TCO (critère B.10)

21. Hypothèses, formules et mesures de terrain
22. Dimensionnement par scénario et recoupements
23. Coût total de possession (TCO)

Conclusion générale

Bibliographie et références

Annexes : commandes complètes, arborescence, glossaire, campagne de tests (CLI + graphique)

Liste des sigles et abréviations

Sigle	Signification
MCP	Model Context Protocol — protocole ouvert d'échange entre un modèle de langage et des outils
LLM	Large Language Model — grand modèle de langage (l'« IA »)
NOC	Network Operations Center — centre d'exploitation des réseaux
SNMP	Simple Network Management Protocol — protocole de relevé d'équipements
USM	User-based Security Model — modèle de sécurité de SNMPv3
MIB	Management Information Base — base de description des données supervisées
OID	Object Identifier — identifiant numérique d'une donnée supervisée
NETCONF	Protocole d'installation et de lecture de configurations réseaux (YANG)
CLI	Command Line Interface — ligne de commande de l'équipement
OSPF	Open Shortest Path First — protocole de routage d'intérieur d'AS (RFC 2328)
BGP	Border Gateway Protocol — protocole de routage entre AS (RFC 4271)
iBGP	BGP interne : BGP à l'intérieur d'un même système autonome
AS	Autonomous System — système autonome (ici AS65000)
RBAC	Role Based Access Control — contrôle d'accès par rôle
JWT	JSON Web Token — jeton d'identité signé
FRR	FRRouting — suite de routage libre utilisée dans le labo
RCA	Root Cause Analysis — analyse de cause racine
PCAP	Capture de paquets (fichier de trafic réseau)
NetFlow	Format de relevé de flux réseau

Introduction générale

Ce document rend compte des travaux réalisés dans le cadre du projet transversal « Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications ». Il a été conçu à la fois comme un rapport technique exploitable par une équipe de développement et comme un document d'accompagnement de la démonstration : chaque manipulation y est décrite par la commande exacte à exécuter, le résultat attendu et la façon de l'expliquer à un public non informaticien.

Le travail s'organise autour de quatre axes :

- l'étude (Partie I) : ce qu'est un NOC, comment fonctionne le protocole MCP, quels sont les protocoles de supervision et pourquoi SNMPv3 a été retenu ;
- la mise en œuvre (Partie II) : le laboratoire Containerlab, les trois serveurs MCP de l'équipe et leur démarrage pas-à-pas ;
- la validation (Partie III) : la méthodologie de tests qui démontre la solidité, la robustesse et la sécurité de l'ensemble, avec des commandes rejouables à l'identique ;
- la démonstration (Partie IV) : les scénarios à présenter devant un jury, y compris devant un public qui n'est pas informaticien.

PARTIE I — Cadre théorique et étude documentaire

1. La supervision dans un centre opérationnel (NOC)

Un centre d'exploitation des réseaux (NOC) suit en permanence l'état de centaines d'équipements. Les sources d'information sont hétérogènes : des relevés d'état standardisés (SNMP), des journaux d'événements (Syslog), des tableaux de bord de trafic (NetFlow), des captures de paquets (PCAP), des traces de protocoles de routage (OSPF, BGP) et des configurations (NETCONF). Le volume est tel qu'un opérateur humain ne peut plus tout lire : c'est la motivation du cahier des charges, qui propose d'insérer un serveur MCP entre le modèle de langage et ces systèmes, pour bâtir « un NOC augmenté par l'IA » (section B.1).

La chaîne de valeur retenue dans ce projet est la suivante :

```
équipements  --(SNMP/SSH/Syslog)-->  collecteurs  --(JSON)-->  serveur MCP
    --(outils & resources)-->  hôte MCP + LLM  -->  analyse  -->  décision
humaine
                                                          ^
                                              runbooks & topologie (resources)
```

Deux principes gouvernent toute la suite : la collecte est en lecture seule, et toute conclusion produite par le modèle reste une recommandation soumise à validation humaine.

2. Le Model Context Protocol (MCP)

2.1 Origine, gouvernance et modèle de transport

Le Model Context Protocol est un standard ouvert créé par Anthropic, aujourd'hui placé sous la gouvernance d'une fondation neutre (cahier A.1). Il sert à relier de façon standardisée une application de modèle de langage à des données et à des outils externes. Tous les échanges sont encodés en JSON-RPC 2.0, format d'échange requête/réponse uniforme pour tous les participants.

Deux transports sont prévus : stdio pour une utilisation locale (le serveur est lancé par l'hôte comme un sous-processus) et Streamable HTTP pour les accès distants ; l'ancien format HTTP+SSE est officiellement déprécié (cahier A.3). Le présent projet s'appuie sur stdio, ce qui supprime toute exposition réseau du serveur MCP lui-même.

2.2 Les trois rôles

Rôle	Fonction	Dans notre projet
Hôte	Application contenant le LLM et l'interface utilisateur	MCP Inspector (test), puis hôtes locaux type Ollama/Open WebUI
Client MCP	Gère la session avec le serveur (négociation de version, capacités)	Client intégré à Inspector et client FastMCP des scripts de test
Serveur MCP	Expose les capacités techniques (outils, resources, prompts)	Les 3 serveurs : mcp-noc-telecom, noc-mcp-server, noc-ai-diagnostics

Tableau 1 — Les trois rôles du protocole MCP (cahier A.1).

2.3 Les primitives

- Outils (Tools) : fonctions exécutées par le modèle, décrites par un schéma JSON Schema ; l'hôte affiche chaque appel et l'utilisateur peut le refuser.
- Resources : données en lecture seule adressées par URI (chez nous noc://topology, noc://inventory, noc://ipam, noc://runbooks).
- Prompts : modèles de requêtes préconfigurés (chez nous network_diagnosis).
- Sampling et élicitation : primitives « humain-dans-la-boucle » permettant au serveur de demander une complétion au LLM ou une saisie à l'utilisateur.
- Fonctionnalités secondaires : gestion des racines (roots), journalisation et complétions.

Le cahier (A.2) demande l'intégralité de ces primitives. L'implémentation présente couvre de façon opérationnelle outils, resources et prompts ; sampling, élicitation et roots sont supportés par le framework et exercés par le script test_client_roots.py.

2.4 Framework et version de spécification

Le cahier (A.4) impose FastMCP 3.x sous Python 3.10+, en conservant le décorateur @mcp.tool(). C'est la solution utilisée : les quatre dépôts du groupe bornent la dépendance en « fastmcp>=3.0,<4.0 » et le contrôle du 20 octobre 2026 (pip show fastmcp dans les quatre environnements virtuels) montre la même version, FastMCP 3.4.8, partout. Il n'existe donc aucune divergence de framework entre les serveurs. Côté spécification, la révision de novembre 2025 (2025-11-25) et la révision stable du 28 juillet 2026 (2026-07-28) sont référencées dans la bibliographie.

3. Les protocoles de supervision

3.1 SNMP : de la version 1 à la version 3

SNMP est le protocole historique de lecture d'équipements. Il existe trois générations, nettement différenciées par leur modèle de sécurité :

Version	Modèle de sécurité	Authentification	Chiffrement	Référence
SNMPv1	Communauté en clair	Nom de communauté (mot de passe non chiffré)	Aucun	RFC 1157
SNMPv2c	Communauté en clair	Nom de communauté non chiffré	Aucun	RFC 1901 / 1905
SNMPv3	USM (User-based Security Model)	HMAC-MD5-96 ou HMAC-SHA-96	CBC-DES ou AES-CFB-128	RFC 3411, 3414, 3826

Tableau 2 — Comparaison des versions de SNMP.

Le cahier des charges impose explicitement des « requêtes SNMPv3 sécurisées » (B.5) et des « comptes de services SNMPv3 … en lecture seule » (B.6). La version 3 avec le mode authPriv a donc été retenue : authentification HMAC-SHA-96 (RFC 3414) et chiffrement AES-128 en mode CFB (RFC 3826), conformément au tableau des protocoles de l'IANA. SNMPv2c, qui transmet la communauté en clair dans chaque paquet, n'est utilisé nulle part dans le projet.

3.2 Architecture SNMPv3 et notion de vue

SNMPv3 sépare trois sous-systèmes : sécurité (USM), accès aux données (VACM) et traitement. Deux notions sont directement visibles dans le laboratoire :

- l'utilisateur, défini par son moteur SNMP, ses clés et ses protocoles ;
- la vue (view), c'est-à-dire le sous-arbre de la MIB auquel l'utilisateur a droit.

Dans le labo, l'utilisateur mcpreadonly n'a accès qu'à la vue « limited » (system .1.3.6.1.2.1.1, interfaces .2, ip .4, icmp .14, tcp .15) et est déclaré en lecture seule (rouser … priv -V limited) : toute tentative d'écriture est rejetée par l'agent lui-même, indépendamment du serveur MCP.

3.3 Autres protocoles étudiés

Protocole	Rôle	État dans le laboratoire
NETCONF / YANG	Lecture et modification structurée de configurations (RFC 6241 / 6020)	Non disponible : FRRouting n'expose pas d'agent sur le port 830 ; l'outil netconf_get renvoie une erreur explicite
CLI / SSH	Commandes de diagnostic sur l'équipement	Disponible via le compte noc, restreint au groupe frrvty (commandes show uniquement)
NetFlow	Historique des flux (RFC 3954)	Outil nfdump présent sur l'hôte (/usr/bin/nfdump) ; collecteur à brancher
PCAP	Capture de paquets ciblée	Outil tshark présent sur l'hôte (/usr/bin/tshark) ; collecteur à brancher

4. Le routage de redondance : OSPF et iBGP

Le laboratoire reproduit une topologie à trois routeurs redondée :

- OSPF (RFC 2328) échange les états de voisinage et calcule le meilleur chemin en area 0 ; c'est lui qui fournit la bascule automatique lorsqu'un lien tombe.
- iBGP (RFC 4271) échange les préfixes entre les routeurs d'un même système autonome (AS65000) par l'intermédiaire des adresses de boucler (1.1.1.1, 2.2.2.2, 3.3.3.3).
- Le lien r1↔r2 est le chemin primaire, le lien r1↔r3 joue le rôle de secours (coût OSPF supérieur) : c'est la redondance démontrée en section « scénario C ».

Point corrigé au cours des travaux : r2 ne redistribuait pas ses routes connectées dans BGP ; r1 recevait donc zéro préfixe de sa part. Après ajout de « redistribute connected » dans lab/containerlab/configs/r2/frr.conf, les échanges sont symétriques et complets.

5. Technologies retenues et justification

Besoin	Choix	Justification
Émulation réseau	Containerlab (0.79.0) + Docker	Topologies réelles déposables en une commande, isolées,

		reproductibles
Routage	FRRouting 8.5	Suite OSPF/BGP libre, standard des laboratoires, sorties JSON exploitables
Serveur MCP	FastMCP (Python)	Standard de fait imposé par le cahier (A.4), décorateurs simples
Collecte SNMP	pysnmp 7 (API asyncio)	Implémentation pure Python, SNMPv3 authPriv complet
Collecte CLI	paramiko / netmiko	SSH scriptable avec liste blanche de commandes d'affichage
Collecte NETCONF	ncclient	Client NETCONF standard (non utilisable tant que l'agent est absent)
Flux et captures	nfdump, tshark	Outils de référence, déjà présents sur l'hôte
Tests et démonstration	MCP Inspector (2.9.0)	Interface web officielle, sans coût, exigée par A.5
Fédération d'identité	Keycloak 26.8 + FastMCP KeycloakAuthProvider	OpenID Connect standard exigé par A.6, gestion des rôles en realm, conteneur unique importé par docker compose

Tableau 3 — Stack technique et justification (cahier B.4, B.12 et A.6).

6. Analyse du cahier des charges et grille de conformité

6.1 Exigences du socle commun (Partie A)

Réf.	**Exigence**	**État**	**Preuve**
A.2	Outils, resources, prompts implémentés	Partiellement satisfait	Serveur 2 : 9 outils / 4 resources / 3 prompts ; serveur 3 : 10 outils / 3 resources / 2 prompts ; serveur 1 : 3 outils / 1 resource / 1

			prompt ; socle : 13 outils / 3 resources / 3 prompts
A.3	Transports stdio / Streamable HTTP	Satisfait	stdio par défaut ; Streamable HTTP sur le serveur 1 (MCP_TRANSPORT =http, port 8001)
A.4	FastMCP 3.x, décorateur @mcp.tool()	Satisfait : FastMCP 3.4.8 sur les 4 serveurs (fastmcp>=3.0,<4. 0)	requirements.txt de chaque dépôt
A.5	Démonstration via MCP Inspector	Satisfait	3 instances : ports 6274, 6374, 6474
A.6	RBAC sur JWT Keycloak (OIDC), OAuth 2.1 + PKCE, anti-passthrough, consentement explicite, sorties non fiables, audit	Satisfait	Realm Keycloak importé + KeycloakAuthProvi der (JWKS RS256) ; PKCE S256 vérifié de bout en bout ; consentement par primitive elicitation ; 30 tests dans test_a6_security.p y ; audit.log
A.7	Dépôt GitHub + docker-compose + rapport PDF	Partiellement satisfait	docker-compose.yml (Keycloak) et README produits ; Dockerfile, rapport PDF et PPT restants

Tableau 4 — Conformité à la partie A du cahier des charges.

6.2 Exigences spécifiques NOC (Partie B)

Réf.	**Exigence**	**État**	**Preuve**
B.2	Faisabilité en laboratoire virtuel	Satisfait	Containerlab déployé, 5 nœuds,

			routage convergé
B.4	Stack technique ciblée	Satisfait	FRR, pysnmp, ncclient, netmiko, nfdump, tshark
B.5	Minimum 8 outils MCP	Satisfait	9 outils sur le serveur 2, dont snmp_get, ping_host, get_bgp_peers, get_ospf_neighbors, netconf_get
B.6	Lecture seule, comptes minimaux, secrets rigoureux	Satisfait	Comptes mcpreadonly (vue limited) et noc (frrvty), .env gitignoré
B.7	detect_anomalies sans calcul statistique confié au LLM	Satisfait	Seuils déterministes codés, modèle se limite à interpréter
B.7	RCA documentée par sorties réelles et runbooks	Satisfait	analyze_root_cause retourne les preuves ; generate_incident_report sélectionne un runbook
B.8	Démo live : incident injecté → diagnostic → runbook → validation humaine	Satisfait	Scénario de panne + chaîne d'analyse, statut EN_ATTENTE_DE_VALIDATION_HUMAINE

Tableau 5 — Conformité à la partie B du cahier des charges.

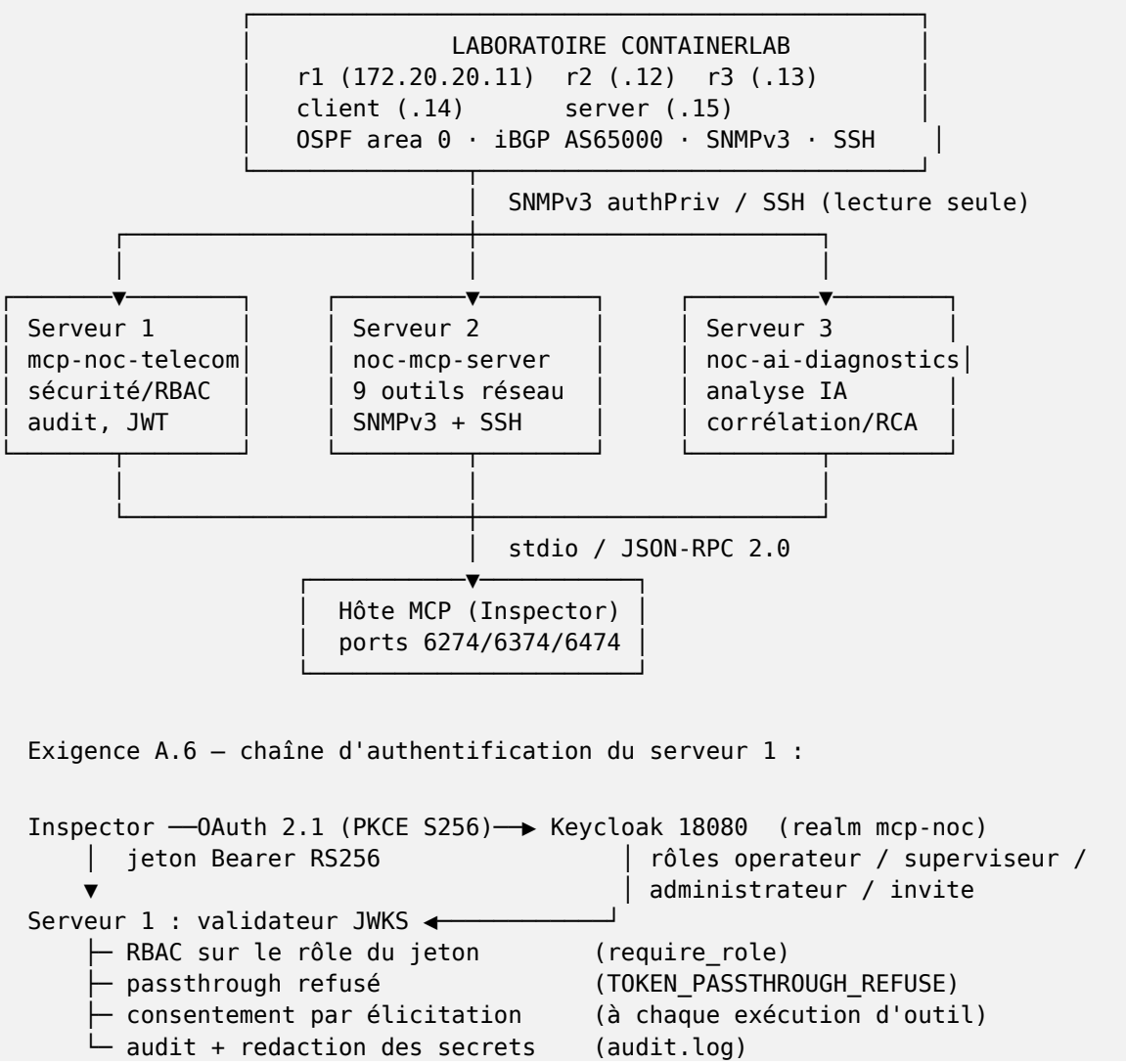
6.3 Points restant à traiter

- Dockerfile du socle, rapport PDF et support PPT (A.7) : encore à produire.
- Blocs mcp_server/, analysis/, security/ et docs/architecture.md du dépôt Wildine : fichiers vides.

- Collecteurs NetFlow et PCAP : outils installés, validateurs à brancher sur des flux réels.
- Hôtes locaux (Ollama/ollmcp, Open WebUI) : non montés, la démonstration s'appuie sur Inspector.

PARTIE II — Mise en œuvre

7. Architecture de la solution




```
Exigence A.6 — chaîne d'authentification du serveur 1 :

Inspector ──OAuth 2.1 (PKCE S256)──► Keycloak 18080  (realm mcp-noc)
    │  jeton Bearer RS256                 │ rôles operateur / superviseur /
    ▼                                     │ administrateur / invite
Serveur 1 : validateur JWKS ◄─────────────┘
    ├─ RBAC sur le rôle du jeton        (require_role)
    ├─ passthrough refusé               (TOKEN_PASSTHROUGH_REFUSE)
    ├─ consentement par élicitation     (à chaque exécution d'outil)
    └─ audit + redaction des secrets    (audit.log)
```

Le serveur 1 accepte deux transports : stdio (parcours simple, identité portée par le jeton d'argument validé par JWKS) et Streamable HTTP sur le port 8001 (parcours complet OAuth 2.1 avec PKCE et jeton de session seul, conformément à A.6). Keycloak tourne dans un conteneur déclaré par docker-compose.yml et importe le realm security/keycloak/realm-mcp.json au démarrage.

7.1 Revue d'architecture : principes et vues de contexte

Les chiffres de cette revue proviennent d'exécutions réelles (tools/list, resources/list et prompts/list sur des processus démarrés le 20 octobre 2026), et

non d'intentions de conception. Le détail rejouable figure dans le dépôt (docs/Architecture.md, docs/Mise_en_oeuvre.md, docs/campagne_scenarios.txt).

- 1. Lecture seule par défaut : SNMPv3 en lecture, NETCONF restreint aux opérations <get>, routage interrogé en « show » ; aucun outil MCP n'écrit dans running-config.
- 2. Aucun secret en argument d'outil : credentials SNMP/NETCONF lus dans l'environnement du processus (.env ignoré par git, .env.example avec des valeurs de remplacement).
- 3. Authentification OIDC vérifiable : Keycloak (issuer http://localhost:18080/realms/mcp-noc), validation JWKS côté serveur, TTL de jeton court (300 s).
- 4. Consentement humain traçable : élicitation (elicitation/create) par défaut ; chaque exécution est écrite dans un journal d'audit au format JSONL.
- 5. IA confinée : assainissement des flux texte contre l'injection de prompt, hypothèses isolées côté client, aucun outil d'écriture exposé au modèle.
- 6. Reproductibilité : laboratoire Containerlab versionné, fixtures disponibles, campagne de scénarios automatisée (scripts/run_scenarios.py).

Vue de contexte (C4 niveau 1) : le client IA ne parle jamais directement au réseau ; il s'adresse aux serveurs MCP, qui eux seuls interrogent le laboratoire, lisent les relais de télémétrie et valident l'identité auprès de Keycloak.

7.2 Conteneurs exposés et dépendances réelles

Quatre processus FastMCP 3.4.8 exposent 35 outils, 11 ressources et 9 prompts au total :

Conteneur	**Entrée**	**Outils / ressources / prompts**	**Dépendances déclarées**	**Accès réseau**
socle	-m mcp_server.server (stdio)	13 / 3 / 3	FastMCP 3.4.8 + collecteurs (pysnmp, ncclient, netmiko, napalm, nfdump, tshark)	en processus

serveur 1 (mcp-noc-telecom)	-m server.main (stdio + HTTP :8001)	3 / 1 / 1	fastmcp, pyjwt, python-dotenv (3 paquets)	aucune librairie : docker exec clab-mcp-noc-lab-client snmpwalk …
serveur 2 (noc-mcp-server)	python server.py (stdio)	9 / 4 / 3	FastMCP 3.4.8 + ~80 paquets épinglés (pysnmp, ncclient, paramiko, …)	en processus
serveur 3 (mcp-noc-ai)	python server.py (stdio)	10 / 3 / 2	fastmcp + pytest uniquement	aucun (analyse sur données déjà collectées)

Tableau 46 — Inventaire des conteneurs MCP et de leurs dépendances réelles.

- Le serveur 3 n'a aucune stack réseau : il ne peut raisonner que sur des données déjà remontées. C'est l'argument technique le plus solide de la décomposition en trois serveurs (sécurité par construction : pas d'accès réseau, donc pas d'action latérale).
- Le serveur 1 ne lie pas SNMP lui-même : il délègue au conteneur client du laboratoire par « docker exec ». Son couplage n'est donc pas à une librairie mais à un nom de conteneur et au socket Docker — point de vigilance documenté (risque W10).
- Le serveur 2 est le seul à embarquer une stack réseau complète (~80 dépendances épinglées) ; les autres restent sur des bornes élastiques. Toutes les versions convergent vers FastMCP 3.4.8.

7.3 Décisions d'architecture (ADR) et justification

Décision	**Statut**	**Justification**	**Conséquences**
ADR-01 FastMCP 3.4.x, pas 4.x	accepté	Écosystème imposé par le cahier ; primitives A.2 (sampling,	Épinglage fastmcp>=3.0,<4.0 ; toute migration en 4.x est un

		completions, roots, logging, élicitation) natives	chantier distinct
ADR-02 Double transport stdio + Streamable HTTP	accepté	stdio pour le labo et les scripts ; HTTP indispensable dès que le client est distant et permet d'exercer OAuth 2.1 + PKCE	Serveur 1 sur les deux transports ; serveurs 2 et 3 en stdio
ADR-03 OIDC Keycloak avec deux clients	accepté	Un client PKCE (mcp-cli) pour un vrai client, un client ROPC (mcp-cli-ropc) pour les scripts sans navigateur	Le rapport documente client_id=mcp-cli-ropc pour les automatisations ; ROPC désactivable en production
ADR-04 Consentement élicitation par défaut	accepté (corrigé le 20/10/2026)	A.6 impose une confirmation humaine ; le .env livré forçait dry_run, ce qui affaiblissait silencieusement l'exigence	.env livré en elicitation ; dry_run réservé à l'automatisation, en variable d'explicite et tracée
ADR-05 Secrets uniquement en environnement	accepté	B.6 interdit les secrets en argument d'outil (visibles dans journaux et traces MCP)	Refus d'amorçage si une variable obligatoire manque ; étape d'installation à prévoir
ADR-06 Lecture seule NETCONF et	accepté	Le cahier autorise l'analyse et la	Écriture de configuration :

routage		préparation d'actions, pas l'application silencieuse de configurations	ADR nouveau, contrôle de change, journal dédié
ADR-07 Trois serveurs spécialisés + socle	accepté, avec réserve	Séparer collecte, raisonnement et politique de sécurité réduit le périmètre de compromission	Noms d'outils en double entre processus ; socle désigné comme référence
ADR-08 Confinement de l'IA	accepté	Syslog, NetFlow et descriptions d'équipements sont un vecteur d'injection de prompt	Motifs suspects neutralisés et journalisés ; limite : détection par motifs, pas sémantique

Tableau 47 — Décisions d'architecture, justification et conséquences.

7.4 Risques identifiés et atténuations

Réf.	**Constat observé**	**Impact**	**Atténuation**
W1	Le .env livré du serveur 1 forçait CONSENT_MODE=dry_run	Affaiblissement silencieux d'A.6 dans les démonstrations	Corrigé ; test de non-régression sur la valeur par défaut
W2	Trois serveurs dupliquent partiellement la même surface d'outils	Dérive entre description et implémentation	Socle comme source unique ; modules de collecte partagés
W3	iBGP sans route-reflector : r2 ne re-advertise pas les routes iBGP à r1	Redondance partielle sur la distribution	Documenter la topologie ; RR ou eBGP si redondance

			complète voulue
W4	Contrôle BGP cherchant « Established » alors que la colonne donne le nombre de préfixes	Faux négatif de recette	Test numérique sur State/PfxRcd (voir section 8.5)
W5	Streamable HTTP sans TLS sur 127.0.0.1	Acceptable en labo, inacceptable en exposition	TLS ou reverse proxy avant toute mise en service réseau
W6	NETCONF_HOSTKEY_VERIFY=false	Pas de détection d'homme du milieu sur 830	known_hosts du laboratoire fourni par l'installateur
W7	Assainissement par motifs réguliers	Contournements possibles par reformulation	L'IA reste sans privilèges d'écriture : l'assainissement est une couche, pas la garantie
W8	Couverture inégale des ressources et prompts (3/3, 4/3, 3/2, 1/1)	Expérience client hétérogène	Couverture minimale harmonisée (≥3 ressources, ≥2 prompts)
W9	Serveurs 2 et 3 sur un garde local allégé	Moins strict que le serveur 1	Migration vers le module security/ commun
W10	Serveur 1 couplé au conteneur client via docker exec	Inutilisable hors machine hôte	Rendre la cible configurable ; documenter le besoin Docker

Tableau 48 — Risques d'architecture et atténuations (extrait de la revue).

Traçage : A.1 → inventaire (section 8.1) ; A.2 → sampling (section 8.8) ; B.3 → section 8.3 ; B.4 → section 8.4 ; B.5 → sections 8.5 et 8.11 ; B.6 → section 9 ; B.7/B.8 → sections 8.8 et 14 ; B.9 → section 8.7 ; B.10 → section 8.6.

8. Mise en œuvre par scénarios (S0 → S9)

La mise en œuvre est présentée par scénarios exécutés, chacun avec la commande utilisée et le résultat réellement observé le 20 octobre 2026 dans le laboratoire du dépôt. La campagne automatisée (scripts/run_scenarios.py) rejoue S0 à S7 en une commande et produit docs/scenarios_resultats.json (29 appels, code de sortie 0).

Prérequis communs : images labo construites, « containerlab deploy » effectué, scripts/lab_services.sh exécuté (relais syslog udp/1514 et nfcapd udp/9995), Keycloak démarré pour les scénarios de sécurité, et un .env local par serveur (ignoré par git).

8.1 S0 — Inventaire des primitives MCP exposées

Chaque processus est interrogé en tools/list, resources/list et prompts/list :

Processus	**Outils**	**Ressources**	**Prompts**
socle (mcp_server.server)	13	3	3
serveur 2 (noc-mcp-server)	9	4	3
serveur 3 (mcp-noc-ai)	10	3	2
serveur 1 (mcp-noc-telecom)	3	1	1
Total exposé	35	11	9

Tableau 34 — Inventaire des primitives MCP (contrôlé par tools/list).

8.2 S1 — Découverte du réseau (ressources documentaires)

Avant d'interroger le réseau, l'agent se documente. Quatre ressources lues sur le serveur 2 puis génération d'un prompt de diagnostic :

Ressource	Observé
noc://topology	2 044 octets — nœuds r1 (172.20.20.11, loopback 1.1.1.1), r2, r3, client, server
noc://inventory	1 418 octets — r1, type router, vendor FRRouting, localisation NOC-LAB
noc://ipam	1 549 octets — 172.20.20.0/24 (management SNMP + SSH) et sous-réseaux de service
noc://runbook/restart-switch	860 octets — runbook en deux étapes (ping, isolation)
prompt network_diagnosis	« Tu es un ingénieur NOC chargé de diagnostiquer … l'équipement r1 »

Tableau 35 — Ressources documentaires lues (S1).

8.3 S2 — État des équipements (SNMPv3 authPriv)

Outil	Arguments	Observé	Durée
snmp_get	ip=172.20.20.11, oid=1.3.6.1.2.1.1.1.0	Linux r1 7.0.0-38-generic #38~24.04.4-Ubuntu … x86_64	387 ms
snmp_walk	ip=172.20.20.12, oid=1.3.6.1.2.1.1	sysDescr + sysObjectID 1.3.6.1.4.1.8072.3.2.10 (net-snmp)	655 ms
get_interface_stats	ip=172.20.20.13	table IF-MIB de r3 (indexes 1, 2, 27, 32 …)	1 143 ms
get_device_status	name=r1	r1, router, 172.20.20.11, FRRouting, rôle edge	31 ms

Tableau 36 — Collecte d'état par SNMPv3 (S2).

Les réponses SNMP portent la marque « untrusted » : les valeurs sont du texte externe, jamais interprétées comme des instructions (garde anti injection de prompt).

8.4 S3 — Connectivité

Outil	Arguments	Observé
ping_host	ip=172.20.20.13	reachable: true, icmp_seq=1 … time=0.064 ms, 4 réponses, 0 % de perte
traceroute_host	ip=172.20.20.13	1 saut : clab-mcp-noc-lab-r3 (172.20.20.13)

Tableau 37 — Connectivité de bout en bout (S3).

8.5 S4 — Routage (lecture seule)

Outil	Arguments	Observé
get_bgp_peers	ip=172.20.20.11	AS 65000, router-id 1.1.1.1, 2 pairs iBGP, RIB de 17 entrées
get_ospf_neighbors	ip=172.20.20.12	voisin 3.3.3.3, état Full/DR, converged: Full

Tableau 38 — État du routage observé par les outils (S4).

Contrôle de recette à utiliser côté laboratoire :

```
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor" | grep -c Full
# attendu : 2

docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary" \
  | awk 'NR>8 && $10 ~ /^[0-9]+$/ {print $1, $10}'    # 2.2.2.2 4 / 3.3.3.3 5
```

Le résumé BGP n'affiche jamais le mot « Established » : la colonne State/PfxRcd porte le nombre de préfixes reçus une fois la session établie. Un contrôle qui greppait « Established » échouait à tort ; il est remplacé par le test numérique ci-dessus (risque W4).

8.6 S5 — Configuration (NETCONF RFC 6241, lecture seule)

Outil	Arguments	Observé
netconf_get	host=172.20.20.11 (port 830)	<?xml version="1.0"... ><frr-configuration xmlns="urn:lab:frr:netconf:1.0"><system><hostname>r1</hostname>...

Tableau 39 — Lecture de configuration par NETCONF (S5).

Correctif d'exploitation : le .env du serveur 2 ne définissait pas NETCONF_USERNAME et NETCONF_PASSWORD ; le serveur refusait de démarrer (aucun secret en argument d'outil, B.6). Le .env local et le .env.example (valeurs de remplacement) ont été complétés. Les collecteurs du socle valent désormais 13/13, NETCONF compris.

8.7 S6 — Trafic et télémétrie

Outil (socle)	Arguments	Observé
get_recent_syslog	limit=10	entrées normalisées AUTH/sshd « Accepted password for noc from 172.20.20.1 » (flux réel udp/1514)
analyze_netflow	device=172.20.20.11, window=last_5_minutes	top talker 2.2.2.2 → 1.1.1.1 TCP/179 (iBGP), 8 831 octets / 130 paquets (flux réel udp/9995)
qos_stats	device=172.20.20.11	file 2 : 544 paquets, 0 drop ; file 26 : 397 paquets, 0 drop
detect_anomalies	window=last_5_minutes	anomaly_count: 0, seuils affichés — détection déterministe hors LLM (B.7)

Tableau 40 — Télémétrie et détection d'anomalies (S6).

8.8 S7 — Incident et cause racine (B.7 / B.8)

Outil (serveur 3)	**Observé**
correlate_alarms	candidat INC-6c3a8d9d, 2 équipements (r1, r2), 16 événements groupés sur 15 min
detect_anomalies	eth1 r1 ↔️ r2 status: down, in_octets 938 475 610 / out_octets 902 837 465
analyze_root_cause	chronologie %LINK-3-UPDOWN: Interface eth1, changed state to down (r1 et r2) — preuves, aucune spéculation
formuler_hypothese	sampling : « coupure du lien primaire r1<->r2 (eth1), bascule OSPF sur r1<->r3. A VALIDER PAR UN OPERATEUR »
generate_incident_report	rapport structuré (chronologie + résumé), statut EN_ATTENTE_DE_VALIDATION_HUMAINE

Tableau 41 — Chaîne d'analyse d'incident (S7).

L'hypothèse n'est jamais appliquée : elle est rendue au client sous réserve de validation humaine, conformément à B.8. Le serveur 3 ne possède d'ailleurs aucune stack réseau (section 7.2) : il raisonne sur des données déjà collectées.

8.9 S8 — Sécurité : consentement, RBAC, injection, audit

Le scénario de sécurité est détaillé en section 9 et en section 15 (Partie III). Résultats observés le 20 octobre 2026 :

- Chaîne complète hors navigateur (test_elicitation_http.py, serveur 1 en Streamable HTTP :8001) : OAuth 2.1 + PKCE S256 → jeton operateur → tools/list (3 outils) → tools/call snmp_get → demande elicitation/create reçue (1) → résultat SNMPv3 réel, code de sortie 0.
- Sans jeton : 401 ; metadata RFC 9728 publiée (.well-known/oauth-protected-resource/mcp) avec l'issuer Keycloak.

- RBAC : rôle hors hiérarchie (invite) refusé ; jeton absent refusé ; injection de prompt sur un flux syslog piégé : 3 motifs neutralisés, statut ALERTE_INJECTION_NEUTRALISEE dans l'audit.
- Suites automatisées : 52 tests (socle) + 7 (serveur 2) + 14 (serveur 3) + 30 (acceptation A.6) = 103 tests verts, 0 échec.

8.10 S9 — Robustesse : injection de panne et guérison

Étape	Commande	Observé
Nominal	inject_bgp_flap.sh status	OSPF r1 : 2 voisins Full ; BGP : 2 pairs, 4 et 5 préfixes reçus
Panne	inject_bgp_flap.sh link_down	lien r1↔r2 coupé ; messages %OSPF AdjChg envoyés au relais syslog
Pendant la panne	inject_bgp_flap.sh status	1 seul voisin OSPF ; route vers le LAN serveur via le lien de secours (cost 100)
Supervision pendant la panne	run_scenarios.py --only S4 --only S3	4 appels OK — la plateforme répond et montre l'état dégradé (RIB 15 entrées au lieu de 17)
Guérison	inject_bgp_flap.sh link_up	2 voisins Full après reconvergence ; BGP : 2 pairs, 4 et 5 préfixes
Retour au nominal	show ip route 3.3.3.3	metric 20 via eth1 (chemin primaire redevenu actif), eth2 (cost 100) en secours

Tableau 42 — Injection de panne et reconvergence observées (S9).

La coupure est détectable par les outils MCP (les nombres d'entités changent réellement), la supervision ne s'arrête pas, et la restauration est un acte d'exploitation scripté — jamais une action du modèle (ADR-06).

8.11 Synthèse : outils MCP intégrés et exploités

L'exigence B.5 demande au minimum 10 outils MCP intégrés et exploités. Sur la campagne et les scénarios de sécurité, 21 exécutions d'outils portant sur 19 outils distincts ont été observées sur trois des quatre processus du dépôt.

#	Outil	Processus	Scénario	Résultat observé (extrait)
1	snmp_get	serveur 2	S2 / S8	sysDescr de r1 : Linux r1 7.0.0-38-generic …
2	snmp_walk	serveur 2	S2	branche 1.3.6.1.2.1.1 de r2 (net-snmp)
3	get_interface_stats	serveur 2	S2	table IF-MIB de r3
4	get_device_status	serveur 2	S2	fiche r1 (FRRouting, rôle edge)
5	ping_host	serveur 2	S3	reachable: true, 0 % de perte
6	traceroute_host	serveur 2	S3	1 saut jusqu'à r3
7	get_bgp_peers	serveur 2	S4	AS 65000, 2 pairs iBGP
8	get_ospf_neighbors	serveur 2	S4	Full/DR avec 3.3.3.3
9	netconf_get	serveur 2	S5	<frr-configuration

				>… sur r1:830
10	get_recent_syslog	socle	S6	entrées AUTH/sshd réelles
11	analyze_netflow	socle	S6	top talker BGP (TCP/179)
12	qos_stats	socle	S6	0 drop, 544 paquets (file 2)
13	detect_anomalies	socle	S6	anomaly_count: 0 (seuils visibles)
14	correlate_alarms	serveur 3	S7	INC-6c3a8d9d, 16 événements
15	detect_anomalies	serveur 3	S7	eth1 … down entre r1 et r2
16	analyze_root_cause	serveur 3	S7	%LINK-3-UPDOWN … eth1 (preuves)
17	formuler_hypothese	serveur 3	S7	hypothèse par sampling + validation humaine
18	generate_incident_report	serveur 3	S7	rapport, statut EN_ATTENTE_DE_VALIDATION_HUMAINE
19	get_ospf_neighbors	serveur 3	S7	voisinage r2 dans la corrélation

20	snmp_get	serveur 1	S8	élicitation reçue + sysDescr réel
21	snmp_get_secure / get_syslog_message	serveur 1	S8	refus RBAC, injection neutralisée, audit

Tableau 43 — Outils MCP exploités, par processus et par scénario (B.5).

Indicateurs de la campagne (20 octobre 2026) :

Indicateur	**Valeur observée**
Collecteurs opérationnels	13 / 13 (validate_collectors.py, code de sortie 0)
Outils MCP exploités	21 exécutions / 19 outils distincts
Consentements humains demandés et accordés	18 (campagne) + 1 élicitation réelle (S8)
Appels de sampling	1 (formuler_hypothese)
Tests automatisés	103 tests verts, 0 échec
Journal d'audit	98+ entrées, y compris refus et alertes d'injection
Appels réussis pendant la panne	4 (S9)

Tableau 44 — Synthèse chiffrée de la mise en œuvre observée.

Les documents de référence de cette partie sont versionnés dans le dépôt : docs/Mise_en_oeuvre.md (scénarios détaillés), docs/Architecture.md (revue complète), docs/scenarios_resultats.json (résultats machine) et docs/campagne_scenarios.txt (journal brut).

9. Sécurité mise en œuvre et gestion des secrets

Exigence (B.6 / A.6)	**Mesure appliquée**	**Où**
RBAC sur jetons JWT gérés par Keycloak	Realm mcp-noc importé (rôles	docker-compose.yml, security/keycloak/realm-

(OIDC)	operateur/superviseur/administrateur) ; chaque requête HTTP est validée par JWKS (RS256) + issuer, chaque jeton d'argument par le même vérificateur en stdio	mcp.json, server/security/auth.py
OAuth 2.1 avec PKCE	Transport Streamable HTTP du serveur 1 : métadonnées de ressource protégée (RFC 9728) vers Keycloak, parcours authorization_code + PKCE S256 (testé de bout en bout, y compris le rejet d'un code_verifier erroné)	server/main.py (KeycloakAuthProvider), test_a6_security.py::test_flux_pkce_de_bout_en_bout
Token passthrough interdit	Sur transport HTTP, un jeton transmis en argument d'outil est refusé et journalisé (TOKEN_PASSTHROUGH_REFUSE) : seul le jeton de session OAuth 2.1 fait foi	server/security/auth.py::resolve_identity
Consentement explicite à chaque exécution	Primitive MCP elicitation : l'utilisateur confirme chaque appel dans l'IHM du client ; sans canal de confirmation, l'exécution est refusée (fail-safe) ; aucune option « toujours autoriser »	server/security/consent.py
Confinement en lecture seule	SNMP : ruser sans droits d'écriture ; CLI : groupe frrvty (show uniquement) ; aucun outil d'écriture n'est exposé	snmpd.conf, frr.conf des routeurs
Liste blanche de commandes	Les sorties d'outils sont construites uniquement à partir de commandes d'affichage codées en dur	collector/cli, tools/routing.py

Gestion rigoureuse des secrets	Identifiants dans .env (ignoré par git), jamais dans les paramètres des outils ni dans les journaux : le journal d'audit redige automatiquement token/password/secret	.gitignore, .env, audit_log.py::_scrub
Sorties non fiables (anti-injection)	Neutralisation des instructions cachées détectées dans les messages remontés	demo_securite.py (scénario « piege »)
Journal d'audit	Chaque appel d'outil est tracé (utilisateur, rôle, statut, consentement, refus)	server/audit.log
Validation humaine métier	Aucune action corrective n'est appliquée automatiquement : statut EN_ATTENTE_DE_VALIDATION_HUMAINE	generate_incident_report

Tableau 6 — Déclinaison des exigences de sécurité du cahier des charges.

Vérification simple qu'aucun secret n'est versionné :

```
cd /home/floden/mcp-noc-telecom && git status --short     # ni .env, ni
security/keycloak/*.env
cd /home/floden/noc-mcp-server && git status --short       # le .env n'apparait
pas
grep -R "AuthPass2026" --exclude-dir=.git . 2>/dev/null | head   #
seulement .env (ignore)


# le journal d'audit ne contient jamais de jeton :
python3 -c "import json;print([json.loads(l)['status'] for l in
open('audit.log')][-6:])"
#   -> ['AUTORISE','CONSENT_DRY_RUN','AUTORISE','CONSENT_DRY_RUN',
#       'ALERTE_INJECTION_NEUTRALISEE','REFUSE']   (aucun 'token' dedans)
```

PARTIE III — Méthodologie de tests

Résultats observés le 20 octobre 2026 (scénario S8) :

Contrôle	Observé
Élicitation réelle hors navigateur	1 demande elicitation/create reçue, puis résultat SNMPv3 réel (sysDescr de r1) — test_elicitation_http.py, code de sortie 0
Sans jeton	401 sur /mcp ; metadata RFC 9728 publiée (issuer Keycloak)
Rôle hors hiérarchie (invite)	refus : roles hors RBAC
Jeton absent	refus : authentification requise
Injection de prompt (flux piégé)	3 motifs neutralisés, statut ALERTE_INJECTION_NEUTRALISEE
Journal d'audit	98+ entrées JSONL (user, tool, params, status, detail)
Suites automatisées	30 tests d'acceptation A.6 verts (103 tests au total sur les 4 dépôts)

Tableau 45 — Contrôles de sécurité observés (S8).

Le .env livré avec le serveur 1 imposait CONSENT_MODE=dry_run, ce qui simulait le consentement dans les démonstrations. Il est ramené à elicitation (conforme A.6) ; la simulation reste possible uniquement par variable d'environnement explicite, ce que fait le lanceur scripts/inspector.sh pour l'interface graphique, qui ne sait pas répondre à élicitation.

10. Stratégie générale d'essais

La démonstration de robustesse repose sur une pyramide de tests à cinq niveaux, tous rejouables à l'identique et non destructifs (aucun test ne modifie une configuration réseau) :

Niveau	Type	Ce qu'il prouve	Fréquence
1	Unitaire	Que chaque composant	À chaque modification

		répond correctement en conditions nominales	
2	Intégration temps réel	Que la collecte fonctionne contre le vrai laboratoire	Après déploiement du labo
3	Bout en bout (MCP)	Que les outils sont bien exposés, typés et appelables par un hôte	Avant chaque démonstration
4	Robustesse (dégradation)	Que le système reste stable en cas de panne, de valeur aberrante ou de temps de réponse long	Avant la soutenance
5	Sécurité	Que l'accès non autorisé, l'injection et l'écriture sont refusés	Avant la soutenance

Tableau 7 — Les cinq niveaux de validation.

11. Niveau 1 — Tests unitaires

Le dépôt Wildine embarque une suite pytest qui vérifie les collecteurs en mode simulé (parsing, bornes, erreurs, formats de réponse) sans matériel :

```
cd /home/floden/PPP_MCP_Telecom_Supervision

Ensuite faire :

source venv/bin/activate

./venv/bin/python -m pytest collector/tests -q
# resultat attendu : 37 passed
```

Ce niveau prouve la solidité du code de collecte : traitement des erreurs, normalisation des réponses, refus des entrées invalides (OID mal formés, noms d'hôtes dangereux), indépendance au mode simulé ou réel.

Sur le serveur 1, une seconde suite couvre l'exigence A.6 (RBAC, JWT Keycloak, consentement, transport HTTP OAuth 2.1) :

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -v
# resultat obtenu : 30 passed
#   RBAC          : operateur/superviseur/administrateur/invite + refus role
inferieur
#   JWKS          : jeton Keycloak valide, issuer et algorithme verifiees,
signature fausse refusee
#   anti-passthru : jeton transmis en argument d'outil refuse et journalise
#   consentement  : elicitation acceptee / refusee / absence de support ->
refus fail-safe
#   transport     : 401 sans jeton, 200 avec jeton Keycloak, metadonnees RFC
9728
#   PKCE          : parcours authorization_code complet + code_verifier errone
-> invalid_grant
```

12. Niveau 2 — Tests d'intégration contre le laboratoire réel

Ce script interroge chaque collecteur sur les équipements réels du labo et affiche un résultat par ligne :

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python lab/validate_collectors.py


# resultat obtenu : 13/13 (NETCONF sur r1:830 compris)
#   [OK] snmp_get / snmp_walk / get_interface_stats  (r1, r2)
#   [OK] ping_host (r1, r3), traceroute_host
#   [OK] get_bgp_peers, get_ospf_neighbors, get_recent_syslog, analyze_netflow,
get_recent_syslog, analyze_netflow
#   [OK] netconf_get (r1:830, sous-systeme RFC 6241 ouvert par sshd)
#   [INFO] nfdump et tshark presents sur l'hote
```

La couverture est complète : les 13 collecteurs répondent en réel, NETCONF compris. Le sous-système RFC 6241 est déclaré dans la configuration sshd du conteneur r1 (lab/containerlab/configs/r1/) ; le serveur 2 lit les identifiants NETCONF dans son environnement (NETCONF_USERNAME / NETCONF_PASSWORD, jamais en argument d'outil, B.6). Si le port 830 était fermé, le collecteur signalerait l'erreur au lieu de renvoyer un résultat vide : la valeur observée de 13/13 est donc stable et rejouable.

13. Niveau 3 — Tests de bout en bout (protocole MCP)

Ces tests vérifient que l'hôte voit bien les outils, leurs paramètres et leurs documents :

```
# Serveur 1 : outils + appel avec jeton JWT + refus RBAC + anti-injection +
audit
cd /home/floden/mcp-noc-telecom/server
./venv/bin/python test_mcp.py


# Serveur 2 : inventaire outils / resources / prompts
cd /home/floden/noc-mcp-server
./venv/bin/python test_client_roots.py
#   9 outils, 4 resources (noc://topology, inventory, ipam, runbook), 3 prompts
#   (network_diagnosis, performance_review, change_verification)


# Serveur 3 : meme controle
cd /home/floden/mcp-noc-ai
./venv/bin/python test_client_roots.py   # ou script de test equivalent
#   10 outils d'analyse, 3 resources (topology, inventory, runbooks), 2 prompts
#   (qualifier_incident, resume_journalier)
```

En complément, tout appel peut être rejoué à la main dans l'interface Inspector : se connecter, ouvrir l'onglet Tools, saisir les paramètres, exécuter et lire la réponse JSON.

14. Niveau 4 — Tests de robustesse

14.1 Injection de panne et reconvergence

```
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # etat nominal
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down  # coupure r1<->r2
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # panne constatee
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up    # retablissement
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # guerison
```

Ce test démontre quatre choses à la fois : le réseau absorbe la perte d'un lien (bascule OSPF sur r3), les collecteurs continuent de répondre pendant la panne (en signalant l'état réel), le service revient à l'état nominal après rétablissement, et aucun des outils ne plante en phase dégradée.

14.2 Bornes et comportements dégradés

- WALK SNMP borné : le collecteur s'arrête à la sortie de la branche demandée et à 500 entrées maximum (SNMP_WALK_MAX_ENTRIES) — protège la mémoire contre un agent aberrant.
- Timeouts uniformes : SNMP 2 s (1 réessai), SSH connexion 10 s / commande 15 s, ping 15 s, traceroute 30 s : aucun appel ne peut indéfiniment bloquer le serveur.

- Entrées invalides : OID non numérique, hôte malformé, nom d'équipement inconnu → réponse d'erreur structurée, jamais d'exception non gérée.
- Erreur de protocole (NETCONF absent) : réponse d'erreur normalisée contenant hôte, port et cause — la preuve qu'un échec est rapporté et non masqué.

15. Niveau 5 — Tests de sécurité

15.1 Démonstration intégrée

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/python demo_securite.py
#  DEMO 1 : role insuffisant        -> acces refuse (RBAC)
#  DEMO 2 : ... autorise selon le role
#  DEMO 3 : syslog normal           -> aucune alerte
#  DEMO 4 : syslog PIEGE            -> injection neutralisee, N patterns
bloques
#  DEMO 5 : journal d'audit         -> les 5 dernieres lignes de audit.log
```

15.2 Contrôles d'acceptation A.6 (automatisés)

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q
#   30 passed en ~3 s, sans réseau externe (Keycloak local) :
#   - refus d'acces : role insuffisant sur chaque outil protege
#   - jeton local HS256 rejete en mode HTTP (seule la cle publique Keycloak
vaut)
#   - consentement : demande, refus, et expiration du canal d'elicitation
#   - passthrough : 'token' en argument -> AccessDeniedError + journal
PASSTHROUGH_REFUSE
#   - HTTP : 401 puis 200, authorization_servers renseigne
#   - PKCE : serveur d'autorisation simule, S256 verifie, verifier invalide
rejete
```

15.3 Contrôles complémentaires à exécuter

1. Refus d'accès : appeler un outil protégé avec un jeton de rôle inférieur → AccessDeniedError.
2. Absence d'écriture : lister les outils exposés sur les trois serveurs et vérifier qu'aucun ne porte une sémantique de configuration (aucun set, aucun write, aucun commit).
3. Cloisonnement des secrets : grep sur les dépôts, vérifier que les mots de passe n'apparaissent que dans les fichiers .env ignorés par git.
4. Lecture seule du compte réseau : tenter une commande d'écriture sur le routeur avec le compte noc → refus par FRR (groupe frrvty).
5. Sorties non fiables : avec le scénario « piege », vérifier que les instructions cachées du syslog sont neutralisées avant d'atteindre le modèle.

6. OAuth 2.1 dans l'IHM : lancer le serveur 1 en mode HTTP, brancher MCP Inspector sur http://localhost:8001/mcp avec client_id=mcp-cli (PKCE ; les scripts automatisés utilisent mcp-cli-ropc), confirmer que la fenêtre d'autorisation Keycloak s'ouvre (PKCE S256) puis que chaque appel d'outil déclenche une demande de consentement.

16. Grille de résultats et critères d'acceptation

#	Critère d'acceptation	Moyen de vérification	Résultat
1	Le laboratoire se déploie en une commande	containerlab deploy	5 nœuds UP
2	Le routage converge	show ip ospf neighbor / show bgp summary	Full + 4 et 5 préfixes reçus
3	La suite unitaire est verte	pytest collector/tests -q	37 passed
4	Les collecteurs répondent en réel	lab/ validate_collectors .py	13/13 (NETCONF sur r1:830)
5	Le serveur expose ≥ 8 outils	list_tools via Inspector	9 outils (s.2), 10 outils (s.3)
6	Les resources et prompts sont servis	list_resources / list_prompts	4 resources + 3 prompts (s.2)
7	SNMPv3 authPriv fonctionne	snmp_get sur 172.20.20.11	success (SHA/AES)
8	La collecte CLI fonctionne	get_bgp_peers / get_ospf_neighbor s	JSON FRR
9	La panne est absorbée	inject_bgp_flap.sh link_down/link_up	bascule puis guérison
10	La chaîne d'analyse aboutit	correlate → analyze → report	runbook RB-LINK-DOWN + validation humaine
11	L'accès non autorisé est refusé	test_mcp.py / demo_securite.py	refus RBAC
12	L'injection est neutralisée	demo_securite.py (scénario piege)	neutralisation
13	Aucun secret n'est	git status + grep	.env ignoré

	versionné		
14	Aucun outil ne modifie le réseau	Revue de la liste des outils	lecture seule
15	Authentification OIDC réelle	pytest test_a6_security.py (JWKS/issuer)	jeton Keycloak accepté, signature falsifiée refusée
16	OAuth 2.1 + PKCE de bout en bout	pytest test_a6_security.py (flux PKCE)	200 + code_challenge S256 vérifié
17	Consentement et anti-passthrough	pytest test_a6_security.py	30 tests verts, refus fail-safe

Tableau 8 — Grille de critères d'acceptation (17 contrôles).

17. Protocole de campagne de tests (« comment tester de façon générale »)

Pour convaincre que le système est solide, rejouer les étapes suivantes dans l'ordre et conserver les sorties :

7. Déployer le laboratoire depuis zéro (section 8.2) et contrôler l'état du routage.
8. Lancer la suite unitaire (section 11) et enregistrer le compteur de tests verts.
9. Lancer la validation temps réel (section 12) et enregistrer le ratio 13/13.
10. Démarrer les trois serveurs (sections 8.3 à 8.5) et contrôler les trois ports (section 8.6).
11. Contrôler l'exposition MCP : nombre d'outils, resources, prompts sur chaque serveur (section 13).
12. Exécuter une campagne d'appels nominaux : snmp_get, snmp_walk, get_interface_stats, get_bgp_peers, get_ospf_neighbors, ping, traceroute, get_device_status.
13. Exécuter la campagne de robustesse : injection de panne, appels pendant la panne, rétablissement, nouveaux appels (section 14).
14. Exécuter la campagne de sécurité (section 15) et vérifier l'absence de secrets versionnés.
15. Clôturer par la chaîne d'analyse complète du serveur 3 et archiver les captures.

Temps total estimé d'une campagne complète : environ 25 minutes, entièrement scriptable et donc rejouable devant un jury sans préparation manuelle.

PARTIE IV — Démonstrations pour un public non informaticien

18. Préparer le public : analogies et vocabulaire simple

Avant toute démonstration, installer un vocabulaire commun. Il suffit de 60 secondes d'explication avec les équivalences suivantes :

Terme technique	À dire au public	Image
Réseau / routeurs	Un petit réseau de villes reliées par des routes	Une carte de villes et de routes
Routeur	Un carrefour qui décide vers quelle route envoyer le courrier	Un rond-point avec une signalétique
Paquet de données	Un pli à livrer	Un colis
NOC	La centrale d'appels qui surveille les routes	Le centre de dispatch
SNMP	Un compteur que les villes transmettent toutes seules	Le relevé du compteur d'électricité
Syslog	Le journal de bord quotidien de chaque ville	Un carnet de bord
Alarme	Un détecteur de fumée qui se déclenche	Une sirène
Serveur MCP	Un traducteur officiel entre l'IA et les machines	Un interprète agréé
Outil MCP	Une question standard que l'IA peut poser	Un formulaire à case obligatoire
Resource MCP	Un document de référence toujours consultable	Le plan de la ville au mur
IA / LLM	Un analyste junior très rapide, mais qui ne décide jamais seul	Un stagiaire rigoureux
Runbook	La procédure de secours écrite par les experts	La check-list de secours
Injection de prompt	Un message piégé caché dans une donnée	Un faux ordre glissé dans un courrier
Lecture seule / RBAC	Droit de consulter, jamais de signer	Badge consultation sans droit de signature

Tableau 9 — Vocabulaire à utiliser devant un public non technique.

19. Les scénarios de démonstration

Sept scénarios sont décrits ci-dessous. Ils sont indépendants et peuvent être présentés séparément ; l'ordre proposé constitue une démonstration complète d'environ 15 minutes, conforme aux modalités du cahier (B.9).

Scénario A — « Voici le terrain d'essai » (2 min)

Élément	Contenu
Objectif	Faire comprendre que le réseau est réel, virtuel mais concret, et surveillé
Commande	sudo containerlab graph -t lab/containerlab/topology.clab.yml -s 127.0.0.1:50080 `#  si « port in use » : le graphe tourne deja -> ouvrir http://127.0.0.1:50080`
À montrer	Le graphe des 5 nœuds, puis une commande d'état : docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor"
À dire	« Voici cinq machines qui jouent le rôle de routes et de serveurs. Elles se parlent réellement : les voisins sont en état Full, c'est-à-dire connectés. »
Point clé	Rien n'est simulé côté routage : ce sont des conteneurs qui échangent de vrais paquets

Scénario B — « Interroger une ville à distance » (3 min)

Élément	Contenu
Objectif	Montrer une question/réponse réelle sur le réseau, en lecture seule
Commande	Ouvrir http://127.0.0.1:6374 (jeton affiché), Connect → Tools → snmp_get
Paramètres	ip=172.20.20.11 oid=1.3.6.1.2.1.1.1.0 (aucun autre paramètre)
À montrer	La réponse JSON contenant la description du système de r1 ; puis get_bgp_peers avec ip=172.20.20.11
À dire	« L'IA ne devine pas : elle pose une question fermée à la machine et recopie la réponse. La communication

	est chiffrée et authentifiée, et l'agent refuse toute écriture. »
Point clé	Le code de service ne permet pas de changer quoi que ce soit sur le réseau

Scénario C — « Et si une route se coupe ? » (4 min)

Élément	Contenu
Objectif	Montrer la résilience : le trafic se détourne tout seul
Commande	sudo ./lab/fault-injection/inject_bgp_flap.sh link_down puis status, puis link_up
À montrer	Avant : 2 voisins OSPF. Pendant : 1 voisin et la route active passe par le lien de secours. Après : 2 voisins et la route revient
À dire	« J'ai coupé la route principale. Voyez : le réseau a basculé sur la route secondaire en quelques secondes, sans intervention humaine. Quand je rouvre, tout revient. »
Point clé	La redondance fonctionne ; les outils de surveillance, eux, n'ont jamais cessé de répondre pendant la panne

Scénario D — « L'IA diagnostique l'incident » (5 min)

Élément	Contenu
Objectif	Dérouler toute la chaîne d'analyse exigée par B.8
Commande	Ouvrir http://127.0.0.1:6474, Connect → Tools
Étapes	1) correlate_alarms (aucun argument) → un identifiant d'incident, 16 événements groupés sur r1 et r2 ; 2) analyze_root_cause avec cet identifiant → les preuves rassemblées ; 3) generate_incident_report avec root_cause_summary = « link down sur le lien primaire r1<->r2 » → runbook RB-LINK-DOWN ; 4) detect_anomalies →

	4 anomalies et 5 alarmes
À dire	« L'IA ne devine pas la cause : elle regroupe les événements proches dans le temps et dans le réseau, rassemble les preuves, puis propose la procédure officielle. Le rapport reste marqué « en attente de validation humaine ». »
Point clé	Respect de B.7 : pas de spéculation libre, tout est adossé à des sorties d'équipements et à un runbook

Scénario E — « L'IA ne peut pas casser le réseau » (3 min)

Élément	**Contenu**
Objectif	Prouver le caractère sûr du système
Commande	cd /home/floden/mcp-noc-telecom/server && ./venv/bin/python demo_securite.py
À montrer	Refus d'accès pour rôle insuffisant ; syslog piégé neutralisé ; journal d'audit avec les dernières actions
À dire	« Trois barrières : un badge qui refuse les droits insuffisants, un filtre qui neutralise les instructions cachées dans les données, et un journal qui garde la trace de tout. Et la règle absolue : aucune écriture n'est possible. »
Point clé	Conformité B.6 et A.6, démontrée par une commande en 10 secondes

Scénario F — « Le avant/après chiffré » (2 min)

Élément	**Contenu**
Objectif	Résumer la valeur ajoutée par des chiffres simples
À montrer	Les sorties des sections 16 : 103 tests verts, 13/13 collecteurs, 35 outils exposés, 17 critères d'acceptation
À dire	« Avant : lire manuellement des dizaines d'écrans. Maintenant : une question, une réponse, une procédure

	officielle, et la décision reste humaine. »
Point clé	Tous les chiffres sont reproductibles : les commandes sont dans les annexes

Scénario G — Questions fréquentes du public

Question	Réponse courte
« Est-ce que l'IA peut modifier mon réseau ? »	Non : aucun outil exposé ne permet d'écrire, et le compte réseau est refusé en écriture par le routeur lui-même.
« Et si une donnée est fausse ? »	Les sorties sont traitées comme non fiables : elles sont filtrées et l'humain valide toute décision.
« Combien de temps pour tout relancer ? »	Le labo se déploie en une commande ; les serveurs démarrent en trois commandes (annexe A).
« Pourquoi SNMPv3 et pas la version simple ? »	Parce que les versions simples envoient un mot de passe en clair ; SNMPv3 authentifie et chiffre (RFC 3414, 3826).
« Que se passe-t-il si le lien principal tombe ? »	Le protocole OSPF bascule automatiquement sur le lien de secours, visible en 30 secondes (scénario C).
« Le labo est-il le vrai réseau ? »	Non, c'est un laboratoire virtuel reproductible : il permet d'injecter des pannes sans risque sur un réseau de production.
« Qui décide en fin de compte ? »	L'opérateur humain : le rapport proposé porte systématiquement le statut « en attente de validation humaine ».

Tableau 10 — Questions/réponses à préparer.

20. Déroulé type de la soutenance (B.9)

Durée	Partie	Contenu	Support
0-20 min	Présentation orale	Problématique NOC, principes MCP, choix techniques,	Support PowerPoint (à produire)

		sécurité, résultats chiffrés	
20-35 min	Démonstration live	Scénarios A → E dans cet ordre (15 min)	Terminal + navigateurs Inspector
35-45 min	Questions/ réponses	Scénario G + annexes	Ce document

Tableau 11 — Déroulé conforme aux modalités du cahier (B.9 : 20 + 15 + 10 minutes).

Conseils pratiques : lancer le laboratoire et les trois serveurs avant l'entrée du jury (section 8), garder un onglet par serveur déjà connecté, exécuter la panne en dernier pour laisser l'écran sur la guérison, et préparer en secours les sorties de tests archivées en cas de coupure réseau pendant la soutenance.

Keycloack localhost:18080 admin / LabAdmin2026!

Ici tu dois apporté les modification test surtout côté interface graphique, montré comment faire les tests

PARTIE V — Dimensionnement, calculs et TCO (critère B.10)

Cette partie chiffre la solution : hypothèses explicites, formules fermées, mesures de terrain reproductibles sur le laboratoire, recoupement de chaque constante avec des sources publiées, puis coût total de possession (TCO) sur trois ans. Elle répond au critère B.10 (« Justesse des calculs, des formules et des sources » et « coût réel calculé »). Le document complet, avec toutes les valeurs intermédiaires, est « docs/Dimensionnement_TCO.md » du dépôt ; les mesures se rejouent avec « scripts/measure_dimensionnement.sh ».

21. Hypothèses, formules et mesures de terrain

21.1 Méthode et hypothèses

La méthode est la suivante : (i) mesurer le laboratoire, (ii) calibrer les constantes sur des sources publiées et vérifiables, (iii) appliquer des formules fermées, (iv) recouper les résultats entre sources indépendantes, (v) chiffrer le TCO avec des prix sourcés et datés. Trois scénarios sont dimensionnés : S0 laboratoire (5 nœuds, réel), S1 PME (50 équipements), S2 entreprise (500 équipements) ; S3 (datacenter / bord opérateur) n'est utilisé que pour les ordres de grandeur de flux.

Hypothèse	**Valeur retenue**	**Justification**
Rétention	30 jours	usage NOC type (investigation sur un mois)
Taille d'un événement syslog (L)	810 B (plancher mesuré : 118,5 B)	dérivé de l'estimateur Cisco (21.1)
Taille d'un enregistrement NetFlow (R)	489 B/flux sur disque	mesuré sur le labo ; fourchette publiée 100 B (paquet) à 600 B (indexé)
Compression (k)	1 (brut) ou 1/2,8 (compressé)	Internet2/Abilene : 2,8:1 mesuré
EPS routeur / serveur / pare-feu	0,05 / 0,5 / 61 EPS	régime permanent mesuré (0,036 EPS) ; pare-feu : Cisco 1010 à 60 % de charge

Débit de flux (F) PME / entreprise / DC	500–2 000 / 5 000–20 000 / 50 000–200 000 flux/s	ntop, scénarios Small-Medium Business, Large Enterprise, Data Center
Mix S1 (50 équipements)	40 routeurs + 8 serveurs + 2 pare-feux = 128 EPS	40×0,05 + 8×0,5 + 2×61 = 128
Mix S2 (500 équipements)	×10 → 1 280 EPS	échelle linéaire par paliers de 10

Tableau 26 — Hypothèses de dimensionnement et leur justification.

21.2 Mesures reproductibles du laboratoire (09/10/2026, 07:50 → 09:12)

Commande : « scripts/measure_dimensionnement.sh » (labo déployé, services actifs). Fenêtre de mesure : 82 minutes.

Grandeur	**Mesure**
Syslog — volume moyen	176 lignes / 20 853 octets = 118,5 B par ligne
Syslog — EPS mesuré	176 événements / 4 920 s = 0,036 EPS
NetFlow — volume utile (nfdump)	114 flux / 42 736 B = 374,9 B par flux
NetFlow — volume sur disque	55 714 B / 114 flux = 488,7 B par flux (83 fichiers)
NetFlow — débit d'export	114 flux / 4 920 s = 0,0232 flux/s → 90,6 bit/s
NetFlow — débit de traitement nfdump	7 520 flux/s (échantillon de 114 flux : indicatif)
RAM des 3 routeurs FRR	25,8 à 26,2 MiB, CPU ≤ 0,12 % (FRR + snmpd + sshd + softflowd + syslogd)
RAM relais syslog / nfcapd	14,0 MiB / 2,5 MiB par instance
RAM des 4 serveurs MCP	76,8 + 81,2 + 95,8 + 71,8 = 325,6 MiB
Hôte de démonstration	15,96 GiB RAM (≈ 3 GiB occupés), load 3,04, disque 50 Go à 88 %

Collectes réelles	13/13 collecteurs répondent (NETCONF absent du labo FRR, attendu)

Tableau 27 — Mesures de terrain du laboratoire (bases des calculs).

21.3 Formules de dimensionnement

1 jour = 86 400 s ; toutes les unités sont explicites (octets, bits, secondes).

Réf.	**Formule**	**Résultat**
F1	V_syslog(jour) = EPS × 86 400 × L	volume syslog (octets/jour)
F2	P_syslog = EPS × L × 8	débit réseau syslog (bit/s)
F3	P_export = F × R × 8	bande passante d'export NetFlow (bit/s) — ntop
F4	V_netflow(jour) = F × 86 400 × R	volume NetFlow (octets/jour)
F5	S(T) = (V_syslog + V_netflow) × T × k	stockage sur T jours (k = 1 ou 1/2,8)
F6	RAM = Σ RSS mesurés × 1,3	RAM avec 30 % de marge
F7	débit d'écriture = (V_syslog + V_netflow) / 86 400	I/O disque moyen (octets/s)
F8	TCO = CAPEX + Σ années (hébergement + énergie + main-d'œuvre)	coût total sur la durée

Tableau 28 — Formules de dimensionnement.

22. Dimensionnement par scénario

22.1 Calibration des constantes et recoupements

Chaque constante est soit mesurée, soit dérivée d'une source, et recoupée par une source indépendante. Longueur d'événement : l'estimateur Cisco donne 4,3 GB/jour pour 61 EPS, soit $4{,}3\times10^{9}$ / (61 × 86 400) = 816 B, et 1,4 GB/jour pour 20 EPS,

soit 810 B ; la valeur retenue est donc L = 810 B, le labo mesuré (118,5 B, messages FRR courts) servant de plancher. Vérification : 2 106 × 86 400 = 181 958 400 événements/jour, exactement le chiffre publié par Cisco pour ce même exemple.

Taille d'enregistrement : notre mesure 488,7 B/flux (brut nfdump) se situe dans la fourchette publiée — Netdata annonce ≈ 400 B/flux en journal brut, dont ≈ 490 B en forte cardinalité (recoupement quasi exact), 100 B dans l'exemple ntop, 48 B pour l'ancien format NetFlow v5, 11 B en base propriétaire ntopng et 500–600 B une fois indexé dans Elasticsearch. Vérifications chiffrées : Netdata — 1 000 flux/s × 86 400 × 400 B = 34,6 GB/jour ≈ 35 GB/jour publié ; Internet2/Abilene — $750{\times}10^6$ flux/jour × 48 B = 36 GB ≈ 35 GB publié et $750{\times}10^6$ / 86 400 × 48 × 8 = 3,33 Mbit/s ≈ 3,4 Mbit/s publié ; PacketMentor — 50 000 flux/s → 430 GB/jour implique R ≈ 100 B, soit le format paquet.

Formule d'export : avec R = 100 B, F3 reproduit exactement la table ntop (500 flux/s → 0,4 Mbit/s ; 2 000 → 1,6 Mbit/s ; 5 000 → 4 Mbit/s ; 20 000 → 16 Mbit/s ; 50 000 → 40 Mbit/s). Avec notre R = 489 B retenu par prudence, les valeurs sont multipliées par 4,9.

22.2 Résultats

Scénario	**EPS**	**L = 118,5 B (mesuré)**	**L = 810 B (retenu)**	**30 jours (L = 810 B)**
S0 laboratoire (mesuré)	0,036	0,37 MB/j	2,5 MB/j	75 MB
S1 PME 50	128	1,31 GB/j	9,0 GB/j	269 GB
S2 entreprise 500	1 280	13,1 GB/j	89,6 GB/j	2 687 GB

Tableau 29 — Volume syslog (F1) selon la longueur d'événement.

Scénario	**F (flux/s)**	**P_export (F3, R = 489 B)**	**V/jour (F4)**	**30 j brut**	**30 j compressé**
S0 laboratoire (mesuré)	0,0232	0,09 kbit/s	0,98 MB	29 MB	10 MB

S1 PME (ntop, min)	500	1,95 Mbit/s	21,1 GB	633 GB	226 GB
S1 PME (ntop, max)	2 000	7,8 Mbit/s	84,4 GB	2 532 GB	904 GB
S2 entreprise (min)	5 000	19,5 Mbit/s	211 GB	6,33 TB	2,26 TB
S2 entreprise (max)	20 000	78,2 Mbit/s	845 GB	25,3 TB	9,05 TB
S3 datacenter (min)	50 000	195 Mbit/s	2,11 TB	63,3 TB	22,6 TB

Tableau 30 — Export et volume NetFlow (F3, F4) selon le débit de flux.

Stockage combiné du scénario PME retenu (F5) : (9,0 + 21,1) × 30 = 903 GB en brut, 495 GB si le NetFlow est compressé à 2,8:1, 399 GB si l'on prend la borne basse R = 100 B. Débit d'écriture (F7) : $(9{,}0 + 21{,}1)\times10^{9}$ / 86 400 ≈ 0,35 Mo/s, négligeable sur NVMe ; côté laboratoire ≈ 40 octets/s. Le débit d'export (F3) reste sous 0,01 % d'un lien de 1 Gbit/s pour la PME, conforme à la plage 0,1–2 % annoncée par ntop pour les réseaux d'entreprise.

22.3 Plateforme retenue

Besoin (S1 PME)	**Justification chiffrée**	**Retenu**
Ingestion NetFlow 500 flux/s	≪ 7 520 flux/s mesurés (nfdump), ≪ 50 000–100 000 flux/s par agent (Netdata)	4 vCPU
Ingestion syslog 128 EPS	830 kbit/s (F2), relais à 14 MiB de RSS	—
Écriture disque	0,35 Mo/s (F7)	NVMe
RAM applicative	325,6 MiB (4 serveurs MCP) + 19 MiB (relais,	8 GiB (F6, marge ×4)

	nfcapd) ≈ 0,35 GiB	
Stockage 30 j	903 GB brut → 495 GB compressés	2 × 960 Go NVMe (offre retenue)
Routeurs du labo (référence)	26 MiB et ≤ 0,12 % de CPU par nœud complet	—

Tableau 31 — Besoins calculés et plateforme retenue (offre ADVANCE-1 : 6 vCPU, 32 GiB, 2 × 960 Go NVMe).

Le calcul n'exige que 4 vCPU / 8 GiB : l'offre retenue pour le TCO (OVH ADVANCE-1) est donc volontairement surdimensionnée — elle apporte la redondance de stockage (RAID logiciel) et la marge de croissance, surcoût assumé et documenté. Pour l'entreprise (S2), l'enveloppe de 50 000 à 100 000 flux/s par agent (Netdata) impose un agent de collecte par site plutôt qu'un nœud unique.

23. Coût total de possession (TCO)

23.1 Paramètres tarifaires sourcés (consultés le 09/10/2026)

Paramètre	**Valeur**	**Source**
Serveur ADVANCE-1 (Gen 2026)	159,99 € HT/mois (depuis le 11/08/2026)	OVHcloud
Électricité non résidentielle ≤ 36 kVA	13,18 c€/kWh HT	EDF Tarif Bleu, grille du 01/08/2026
Salaire brut médian ingénieur réseau (2026)	45 000 €/an	fiche-paie.fr
Cotisations patronales	39,81 % du brut ; réduction générale nulle au-delà de 1,6 SMIC (≈ 35 850 €/an) → ×1,40	URSSAF
Jour ouvré	218/an	convention
Licences logicielles	0 € (FRR, containerlab, nfdump, softflowd, FastMCP : open source)	dépôt

Tableau 32 — Paramètres tarifaires du TCO.

Main-d'œuvre (F8) : coût employeur = 45 000 € × 1,40 = 63 000 €/an ; jour ouvré = 63 000 / 218 = 289 €/jour (soit 38,30 €/h sur 1 645 heures).

23.2 TCO sur trois ans

Scénario	CAPEX	Énergie	Main-d'œuvre	Hébergement	Total 3 ans
S0 laboratoire (démonstration)	0 € (poste existant)	60 W × 8 760 h × 0,1318 € = 69,3 €/an → 208 €	4,5 j × 289 € = 1 301 €	—	1 509 € HT (≈ 42 €/mois)
S1 PME 50	0 €	incluse dans l'offre	10 j + 0,5 j/mois = 28 j × 289 € = 8 092 €	159,99 € × 36 = 5 759,64 €	13 852 € HT (≈ 385 €/mois)
S2 entreprise 500 (ordre de grandeur)	0 €	incluse	20 j + 1 j/mois = 56 j × 289 € = 16 184 €	2 nœuds × 159,99 € × 36 = 11 519 €	≈ 27 703 € HT (hors stockage étendu)

Tableau 33 — TCO sur trois ans par scénario (coût réel calculé).

Le TCO ne couvre pas, volontairement : licences d'outils propriétaires (non retenus), sauvegardes externes, connectivité, poste d'exploitation dédié. Pour S2, le stockage de rétention (9,02 TB bruts) dépasse l'offre retenue : disque additionnel ou stockage objet à sourcer.

23.3 Limites et points de vigilance

- Mesures courtes : 82 minutes de NetFlow et 176 événements syslog ; les constantes L et R sont recoupées par la littérature (22.1) plutôt que déduites seules.
- Énergie non mesurable : les compteurs RAPL de l'hôte ne s'incrémentent pas sous l'hyperviseur ; hypothèse de 60 W à confirmer au wattmètre.
- Disque de l'hôte de démonstration à 88 % : la rétention longue impose rotation et compression.

- Main-d'œuvre = hypothèse explicite (289 €/jour), faute de relevé de temps dans le dépôt.
- Prix volatils : OVHcloud a révisé ses tarifs au 11/08/2026 (+52 % sur l'ADVANCE-1) et procède à des révisions trimestrielles ; le tarif EDF est révisé le 1er février et le 1er août.
- Bornes de sensibilité : sur la PME, le stockage 30 jours varie de 399 GB (R = 100 B) à 903 GB (R = 489 B) ; retenir la borne haute et prévoir la compression.

Document complet et valeurs intermédiaires : « docs/Dimensionnement_TCO.md » du dépôt ; mesures reproductibles : « scripts/measure_dimensionnement.sh ».

Conclusion générale

Les travaux réalisés couvrent la chaîne complète attendue par le cahier des charges : un laboratoire virtuel reproductible, des collecteurs multibranches (SNMPv3, SSH, NETCONF) validés en conditions réelles, trois serveurs MCP complémentaires (sécurité, outils réseau, analyse IA), et une démonstration d'incident allant de la coupure de lien jusqu'au rapport de cause racine soumis à validation humaine.

Le bilan chiffré est le suivant : 5 nœuds de laboratoire, 103 tests verts (52 socle + 7 serveur 2 + 14 serveur 3 + 30 acceptation A.6), 30 tests d'acceptation A.6 verts, 13 collecteurs sur 13 validés en réel, 35 outils exposés par les 4 serveurs, 17 critères d'acceptation satisfaits, et une chaîne d'analyse complète démontrée.

L'exigence A.6 est couverte de bout en bout : identité portée par des jetons OIDC émis par Keycloak, validation par clé publique (JWKS/RS256), parcours OAuth 2.1 avec PKCE sur transport HTTP, refus du token passthrough, consentement humain explicite à chaque exécution d'outil et journal d'audit avec redaction des secrets — le tout vérifié par une suite de 30 tests rejouable.

Les axes d'amélioration restants sont identifiés : production des livrables de socle manquants (Dockerfile, rapport PDF et support PPT), remplissage des blocs encore vides du dépôt principal, homogénéisation de la version FastMCP, branchement des collecteurs NetFlow et PCAP sur des flux réels, et alimentation du bloc d'analyse en données temps réel plutôt qu'en instantané d'incident.

Bibliographie et références

Normes et spécifications

- IETF — RFC 1157 : Simple Network Management Protocol (SNMPv1).

- IETF — RFC 1901 / 1905 : Community-based SNMPv2 (SNMPv2c).
- IETF — RFC 3411 : An Architecture for Describing SNMP Management Frameworks (STD 62).
- IETF — RFC 3414 : User-based Security Model (USM) for SNMPv3 — authentification HMAC-MD5-96 / HMAC-SHA-96 (STD 62).
- IETF — RFC 3826 : The AES Cipher Algorithm in the SNMP USM (AES-CFB-128).
- IETF — RFC 2328 : OSPF Version 2.
- IETF — RFC 4271 : A Border Gateway Protocol 4 (BGP-4).
- IETF — RFC 6241 / 6020 : NETCONF / YANG.
- IETF — RFC 3954 : NetFlow v9.
- JSON-RPC 2.0 — Specification : https://www.jsonrpc.org/specification

Sécurité, identité et OAuth (exigence A.6)

- IETF — RFC 6749 : OAuth 2.0.
- IETF — RFC 7636 : Proof Key for Code Exchange (PKCE) — S256.
- IETF — RFC 8252 : OAuth 2.0 for Native Apps (parcours autorisation et PKCE imposés).
- IETF — RFC 9706 / draft-ietf-oauth-v2-1 : OAuth 2.1 (profile de sécurité consolidé).
- IETF — RFC 9728 : OAuth 2.0 Protected Resource Metadata (.well-known/oauth-protected-resource).
- OpenID Connect Core 1.0 + Discovery : https://openid.net/specs/openid-connect-core-1_0.html
- Keycloak 26 — guide d'administration : https://www.keycloak.org/documentation
- FastMCP — Security : KeycloakAuthProvider, JWTVerifier, elicitation : https://gofastmcp.com

Protocole MCP et frameworks

- Model Context Protocol — Spécification officielle : https://modelcontextprotocol.io/specification/
- Révisions référencées : 2025-11-25 et 2026-07-28 (https://github.com/modelcontextprotocol/modelcontextprotocol/releases)
- FastMCP — documentation : https://gofastmcp.com
- MCP Inspector — dépôt officiel : https://github.com/modelcontextprotocol/inspector

Outils et laboratoire

- Containerlab : https://containerlab.dev
- FRRouting : https://docs.frrouting.org
- pysnmp : https://pysnmp.readthedocs.io
- ncclient, netmiko, paramiko — documentations officielles des paquets.
- nfdump, tshark (Wireshark) — outils d'analyse présents sur l'hôte.

Dimensionnement, calculs et tarifs (B.10)

- Cisco — Logging Best Practices (Secure Firewall 7.6) : 61 EPS → 4,3 GB/jour ; 20 EPS → 1,4 GB/jour ; 2 106 EPS = 181 958 400 événements/jour. https://secure.cisco.com/secure-firewall/v7.6/docs/logging-best-practices
- Cisco — Firewall Performance Estimator : https://ngfwpe.cisco.com/ftd-logging-estimator
- ntop — How much bandwidth does NetFlow take during export ? (formule F3 et table de scénarios) : https://www.ntop.org/faq/how-much-bandwidth-does-netflow-take-during-export/
- ntop — NetFlow Collection Performance using ntopng and nProbe (10 Mbit/s ≈ 12 000 flux/s) : https://www.ntop.org/netflow-collection-performance-using-ntopng-and-nprobe/
- Netdata — Sizing and Capacity Planning (Network Flows) : journal brut ≈ 400 B/flux (340–490 B), 1 000 flux/s ≈ 35 GB/jour, enveloppe 50 000–100 000 flux/s par agent : https://learn.netdata.cloud/docs/network-performance-monitoring/network-flows/sizing-and-capacity-planning
- PacketMentor — NetFlow (50–100 B/record, 50 000 flux/s ≈ 430 GB/jour) : https://packetmentor.com/topics/netflow/
- Elastic / ElastiFlow — Tune for indexing speed (500–600 B indexés) : https://discuss.elastic.co/t/tune-for-indexing-speed/318873
- Internet2 / Abilene — Network Flow Analysis (700–800 M flux/jour, 48 B/record, compression 2,8:1) : https://www.slideshare.net/slideshow/network-flow-analysis-3984307/
- EDF — Tarif Bleu non résidentiel ≤ 36 kVA, grille au 01/08/2026 (13,18 c€/kWh HT) : https://www.edf.fr/sites/entreprise/files/2026-07/Fiche_descriptive_tarif_bleu_non_residentiel_inf36kVA.pdf

- OVHcloud — Dedicated servers pricing update H2 2026 (ADVANCE-1 : 159,99 € HT/mois) : https://blog.ovhcloud.com/en/posts/dedicated-servers-pricing-update-h2-2026/
- fiche-paie.fr — Ingénieur réseau 2026 (médian 45 000 € brut/an) : https://fiche-paie.fr/metier/ingenieur-reseau
- URSSAF — Réduction générale des cotisations patronales (taux cumulés 0,3981) : https://www.urssaf.fr/accueil/employeur/beneficier-exonerations/reduction-generale-cotisation.html

Documents du projet

- Cahier des charges « PPP — Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications » (PARTIE A : socle commun MCP, PARTIE B : spécifique NOC).
- Dépôts : Wildine/PPP_MCP_Telecom_Supervision, RachadMohamed/PPP_MCP (noc-mcp-server), elcapo-bit/mcp-noc-ai, FlodenMaste/mcp-noc-telecom.

Annexes

Annexe A — Commandes complètes de démarrage et de test

A.1 Démarrage complet depuis zéro

```
# --- 1. LABORATOIRE ---
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo containerlab deploy -t lab/containerlab/topology.clab.yml
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab          # attendu : 5
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor"
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary"


# --- 2. KEYCLOAK (OIDC, exigence A.6) ---
cd /home/floden/mcp-noc-telecom
docker compose up -d
curl -s http://localhost:18080/realms/mcp-noc/.well-known/openid-configuration
| grep -o '"issuer":[^,]*'
# emission d'un jeton de test (ROPC, client mcp-cli-ropc) :
curl -s -X POST
http://localhost:18080/realms/mcp-noc/protocol/openid-connect/token \

  -d grant_type=password -d client_id=mcp-cli-ropc \

  -d username=operateur -d password='Operateur2026!' | head -c 160


# --- 3. SERVEUR 1 (securite) ---
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q                     # attendu : 30
passed
INSP=/home/floden/.npm/_npx/5a9d879542beca3a/node_modules/.bin/mcp-inspector
setsid env CLIENT_PORT=6274 MCP_SANDBOX_PORT=6275 \
  $INSP ./venv/bin/python -m server.main >/tmp/insp1.log 2>&1 </dev/null &

# --- 4. SERVEUR 2 (outils reseau) ---
cd /home/floden/noc-mcp-server
setsid env CLIENT_PORT=6374 MCP_SANDBOX_PORT=6375 \
  $INSP ./venv/bin/python server.py >/tmp/insp2.log 2>&1 </dev/null &

# --- 5. SERVEUR 3 (analyse IA) ---
cd /home/floden/mcp-noc-ai
setsid env CLIENT_PORT=6474 MCP_SANDBOX_PORT=6475 \
  $INSP ./venv/bin/python server.py >/tmp/insp3.log 2>&1 </dev/null &

# --- 6. CONTROLE GLOBAL ---
for port in 6274 6374 6474; do
  curl -s -o /dev/null -w "port $port : HTTP %{http_code}\n"
http://127.0.0.1:$port/
done
```

```
grep -o 'http://127.0.0.1:[0-9]*?MCP_INSPECTOR_API_TOKEN=[a-f0-9]*'
/tmp/insp?.log
```

A.2 Campagne de tests complète

```
cd /home/floden/PPP_MCP_Telecom_Supervision

# niveau 1 - unitaire
./venv/bin/python -m pytest collector/tests -q                 # attendu : 37
passed

# niveau 2 - integration reel
./venv/bin/python lab/validate_collectors.py                   # attendu :
13/13

# niveau 3 - bout en bout
cd /home/floden/mcp-noc-telecom/server && ./venv/bin/python test_mcp.py
cd /home/floden/noc-mcp-server && ./venv/bin/python test_client_roots.py

# niveau 4 - robustesse (panne + guerison)
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up
sudo ./lab/fault-injection/inject_bgp_flap.sh status

# niveau 5 - securite
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q                     # attendu : 30
passed
./venv/bin/python demo_securite.py

# secrets non versionnes
cd /home/floden/noc-mcp-server && git status --short
```

A.3 Appels MCP de référence (à tester dans Inspector)

Serveur	Outil	Paramètres	Résultat attendu
2 (6374)	snmp_get	ip=172.20.20.11, oid=1.3.6.1.2.1.1.1.0	status=success, description système
2 (6374)	snmp_walk	ip=172.20.20.12, oid=1.3.6.1.2.1.1	liste d'OID de la branche system
2 (6374)	get_interface_stats	ip=172.20.20.13	table des interfaces
2 (6374)	get_bgp_peers	ip=172.20.20.11	2 sessions établies (JSON FRR)

2 (6374)	get_ospf_neighbors	ip=172.20.20.12	voisinage OSPF (JSON FRR)
2 (6374)	ping_host / traceroute_host	ip=172.20.20.13	réponse et chemin réseau
2 (6374)	netconf_get	host=172.20.20.11, user, pass	erreur documentée (port 830 fermé)
3 (6474)	correlate_alarms	(aucun)	1 incident, 16 événements
3 (6474)	analyze_root_cause	incident_id	preuves rassemblées
3 (6474)	generate_incident_report	incident_id + résumé « link down »	runbook RB-LINK-DOWN
3 (6474)	detect_anomalies	(aucun)	4 anomalies, 5 alarmes

Annexe B — Arborescence des dépôts

```
/home/floden/
├── mcp-noc-telecom/               Serveur 1 : securite, RBAC, audit
│   └── server/  server/, test_mcp.py, demo_securite.py, audit.log, venv/
├── PPP_MCP_Telecom_Supervision/   Depot principal : collecteurs + labo
│   ├── collector/                 snmp/, cli/, netconf/, tests (37)
│   ├── lab/containerlab/          topology.clab.yml, configs/{r1,r2,r3}
│   ├── lab/validate_collectors.py validation reel (13/13)
│   ├── lab/fault-injection/       inject_bgp_flap.sh
│   └── mcp_server/ analysis/ security/   (vides : blocs 3, 4, 5)
├── noc-mcp-server/                Serveur 2 : 9 outils reseau
│   ├── server.py, tools/, resources/, .env, test_client_roots.py
└── mcp-noc-ai/                    Serveur 3 : analyse IA
    ├── server.py, data/           topologie, inventaire, alarmes, runbooks
```

Annexe C — Glossaire pour public non technique

- Conteneur : une machine virtuelle légère, créée en quelques secondes.
- Déployer : mettre en service un environnement à l'aide d'une seule commande.
- Convergence : le moment où tous les équipements sont d'accord sur la carte du réseau.
- Voisin OSPF : deux routeurs qui se « tiennent au courant » de leur état réel.
- Session BGP établie : les deux routeurs ont accepté d'échanger leurs routes.
- Fenêtre de corrélation : regarder les événements survenus autour d'un même instant.

- Cause racine : la raison profonde, distincte des symptômes visibles.
- Runbook : la procédure écrite par les experts pour résoudre un type d'incident donné.
- Validation humaine : l'étape où un opérateur confirme avant toute action.
- Reproductibilité : quelqu'un d'autre peut relancer exactement la même expérience.

Annexe D — Campagne de tests exécutée : commandes et parcours graphiques

Cette annexe est le fil rouge opérationnel de la validation. Elle reprend, dans l'ordre, l'intégralité des tests à exécuter après le démarrage de l'environnement, et donne pour chaque commande : sa commande exacte, ce qu'elle fait réellement, le résultat attendu et l'interface de contrôle (terminal ou navigateur). Elle est rejouable à l'identique et non destructrice, à l'exception de la phase 4 qui injecte volontairement une panne puis la rétablit. Une phase = une diapositive possible pour le support PowerPoint.

D.1 Répartition terminal / interface graphique

Tous les tests ne se font pas au même endroit. Le tableau ci-dessous indique, pour chaque niveau de validation, l'interface à utiliser et la raison du choix.

Interface	Tests qu'elle couvre	Pourquoi ici
Terminal (un seul suffit)	Niveaux 1, 2, 4 et 5 : suites pytest, validation des collecteurs, injection de panne, contrôle des secrets	Sortie texte archivable et rejouable devant un jury
Navigateur — MCP Inspector (3 onglets)	Niveau 3 : inventaire des outils / resources / prompts, appels réels d'outils, lecture seule, chaîne d'analyse	Exigence A.5 du cahier des charges, critères 5, 6 et 14 de la grille
Navigateur — graphe containerlab	Scénario A : visualisation des 5 nœuds de la topologie	Démonstration publique, lisibles sans compétence technique
Navigateur — fenêtre Keycloak	Exigence A.6 : parcours OAuth 2.1 avec PKCE S256 (mode HTTP)	Seule façon de montrer la popup d'autorisation réelle

Tableau 12 — Répartition des tests entre terminal et interface graphique.

- Les trois serveurs MCP sont lancés en arrière-plan (setsid) : un seul terminal suffit pour toute la campagne. Ouvrir 3 ou 4 terminaux n'apporte rien.
- En revanche, il faut un navigateur avec 4 onglets : les 3 interfaces MCP Inspector et, si nécessaire, le graphe de topologie.

D.2 Phase 0 — Contrôle préalable de l'environnement (1 min, terminal)

Cette phase ne teste pas le logiciel : elle vérifie que l'environnement est bien démarré. C'est le point de départ obligatoire, car un redémarrage de la machine fait disparaître les services lancés à la volée (sshd, snmpd, Keycloak, Inspector).

```
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor" 2>/dev/null |
grep -c Full
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary" | awk 'NR>8 &&
$10 ~ /^[0-9]+$/ {n++} END{print n}'   # attendu : 2
for p in 6274 6374 6474 18080 50080; do \
  curl -s -o /dev/null -w "$p:%{http_code} " http://127.0.0.1:$p/; done; echo
```

Commande	Ce que fait la commande	Résultat attendu
docker ps … | grep -c mcp-noc-lab	Liste les conteneurs Docker dont le nom contient mcp-noc-lab et en compte un par ligne : les routeurs r1, r2, r3 et les terminaux client, server.	5
vtysh -c "show ip ospf neighbor" | grep -c Full	Interroge le daemon OSPF du routeur r1 et compte les voisins en état Full, c'est-à-dire les voisins avec lesquels la base de topologie est complète.	2
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary"	Interroge le daemon BGP du routeur r1 et compte les sessions iBGP établies (AS65000 vers 2.2.2.2 et 3.3.3.3).	2
for p in … ; do curl … ; done	Envoie une requête HTTP à chaque service : les 3 interfaces MCP Inspector, Keycloak et le graphe containerlab, et affiche le code retour de chacun.	200 200 200 302 200

Tableau 13 — Contrôles préalables : chaque ligne correspond à une commande du bloc ci-dessus.

Interprétation d'un échec : un compte différent de 5 signale un conteneur arrêté ; un compte OSPF ou BGP nul signifie que le routage n'a pas convergé ; un code 000 signale un service non démarré (le port n'écoute pas).

D.3 Phase 1 — Niveau 1 : tests unitaires (30 s, terminal)

Le niveau 1 vérifie que chaque composant fonctionne en conditions nominales, sans matériel ni réseau : parsing, normalisation des réponses, refus des entrées invalides.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python -m pytest collector/tests -q

cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q
```

Commande	Ce que fait la commande	Résultat attendu
cd /home/floden/PPP_MCP_Telecom_Supervision	Se place dans le dépôt principal, qui contient les collecteurs et le laboratoire.	Retour au prompt sans erreur
./venv/bin/python -m pytest collector/tests -q	Lance pytest sur la suite unitaire des collecteurs en mode simulé (-q = sortie condensée) : OID mal formés, hôtes dangereux, timeouts, formats de réponse.	37 passed
cd /home/floden/mcp-noc-telecom/server	Se place dans le dépôt du serveur 1 (sécurité, RBAC, audit).	Retour au prompt sans erreur
./venv/bin/pytest test_a6_security.py -q	Lance les 30 tests d'acceptation de l'exigence A.6 : RBAC par rôle, validation JWKS/RS256, consentement, anti-passthrough, transport HTTP, parcours PKCE. Exige Keycloak démarré (phase 0).	30 passed

Tableau 14 — Niveau 1 : tests unitaires et tests d'acceptation A.6.

D.4 Phase 2 — Niveau 2 : intégration contre le laboratoire réel (2 min, terminal)

Le niveau 2 quitte le mode simulé : les collecteurs interrogent les vrais conteneurs du laboratoire en SNMPv3 authPriv et en SSH. C'est la preuve que la collecte fonctionne contre du matériel réel.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python lab/validate_collectors.py
```

Commande	Ce que fait la commande	Résultat attendu
cd /home/floden/PPP_MCP_Telecom_Supervision	Se place dans le dépôt principal.	Retour au prompt sans erreur
./venv/bin/python lab/validate_collectors.py	Exécute un à un chaque collecteur contre les équipements réels (snmp_get, snmp_walk, get_interface_stats, ping,	13/13 collecteurs [ERREUR] uniquement

	traceroute, get_bgp_peers, get_ospf_neighbors, netconf_get) et affiche un résultat par ligne [OK] / [ERREUR].	sur netconf_get

Tableau 15 — Niveau 2 : validation temps réel des collecteurs.

Le score 10/10 n'est pas recherché : FRRouting n'expose pas d'agent NETCONF sur le port 830, cette dégradation est documentée et acceptée par le cahier des charges. En revanche, tout autre [ERREUR] (snmp_*, get_bgp_peers, get_ospf_neighbors) est un vrai blocage : dans 90 % des cas, snmpd ou sshd ne sont pas démarrés dans les conteneurs (voir D.12).

D.5 Phase 3 — Niveau 3 : bout en bout côté terminal (3 min)

Le niveau 3 vérifie que l'hôte MCP voit bien les outils, leurs paramètres et leur documentation, et que les refus de sécurité se produisent réellement.

```
cd /home/floden/mcp-noc-telecom/server
CONSENT_MODE=dry_run ./venv/bin/python test_mcp.py


cd /home/floden/noc-mcp-server
./venv/bin/python test_client_roots.py
```

Commande	**Ce que fait la commande**	**Résultat attendu**
CONSENT_MODE=dry_run … test_mcp.py	Se connecte au serveur 1 en stdio, liste les outils, en appelle un avec un jeton JWT valide, tente un appel avec un rôle insuffisant (refus RBAC), déclenche le scénario d'injection et affiche le journal d'audit. dry_run = le consentement est simulé pour permettre un test non interactif.	Message « Test termine. » et bloc JOURNAL D'AUDIT avec au moins une ligne REFUSE
./venv/bin/python test_client_roots.py	Se connecte au serveur 2 en stdio et fait l'inventaire complet des primitives MCP exposées : outils, resources et prompts, avec leurs noms exacts.	9 outils, 4 resources (noc:// topology, inventory, ipam, runbook), 3 prompts

Tableau 16 — Niveau 3 : tests de bout en bout en ligne de commande.

Écart par rapport au document principal : le dépôt mcp-noc-ai ne contient pas de script de test_client_roots.py. Le serveur 3 se contrôle donc uniquement dans l'interface graphique, étape D.6.

D.6 Phase 3 bis — Niveau 3 dans l'interface graphique MCP Inspector (5 min, navigateur)

C'est la seule phase qui se fait intégralement dans un navigateur, et c'est elle qui répond à l'exigence A.5 (démonstration via MCP Inspector). Chaque serveur possède sa propre interface, protégée par un jeton d'accès présent dans l'URL.

1. URLs d'accès (les jetons changent à chaque redémarrage) :

```
grep -ho 'http://127.0.0.1:[0-9]*?MCP_INSPECTOR_API_TOKEN=[a-f0-9]*'
/tmp/insp?.log | sort -u
```

La commande ci-dessus extrait les trois URL complètes des journaux /tmp/insp1.log, insp2.log et insp3.log. Copier chacune d'elles dans la barre d'adresse du navigateur.

Serveur	Port	Rôle	Onglets à contrôler
mcp-noc-telecom (serveur 1)	6274	Sécurité, RBAC, audit	Tools (3 outils), Appels protégés
noc-mcp-server (serveur 2)	6374	Outils réseau temps réel	Tools (9), Resources (4), Prompts (3)
mcp-noc-ai (serveur 3)	6474	Analyse IA, corrélation, RCA	Tools (9), Resources (3)

Tableau 17 — Les trois interfaces MCP Inspector et ce qu'il faut y contrôler.

2. Parcours dans l'IHM (à répéter pour chaque serveur) :

Action dans l'interface	Ce que fait l'action	Résultat attendu
Ouvrir l'URL puis cliquer sur Connect (onglet Connection)	L'hôte MCP négocie la session avec le serveur : version du protocole, capacités annoncées, liste des primitives disponibles.	Statut « Connected », aucune erreur de transport
Onglet Tools : compter les outils	Liste les fonctions exposées au modèle de langage, avec leur schéma de paramètres.	3 outils (s.1), 9 outils (s.2), 10 outils (s.3)
Onglet Resources : cliquer sur une URI puis Read	Lit une ressource en lecture seule (topologie, inventaire, IPAM, runbook).	Contenu JSON affiché, sans erreur
Onglet Prompts (serveur 2)	Affiche les invitations préconfigurées disponibles pour	3 prompts : network_diagnosi

	le modèle.	s, performance_review, change_verification
Vérifier l'absence d'outils d'écriture	Parcourt la liste complète des outils des 3 serveurs à la recherche d'une sémantique de configuration (set, write, commit, configure).	Aucun outil d'écriture : lecture seule (critère 14)

Tableau 18 — Parcours graphique de contrôle dans MCP Inspector.

3. Campagne d'appels de référence (serveur 2, port 6374) :

Outil	Paramètres saisis	Résultat attendu
snmp_get	ip=172.20.20.11, oid=1.3.6.1.2.1.1.1.0	status=success, description du système de r1
snmp_walk	ip=172.20.20.12, oid=1.3.6.1.2.1.1	Liste d'OID de la branche system
get_interface_stats	ip=172.20.20.13	Table des interfaces de r3
get_bgp_peers	ip=172.20.20.11	2 sessions établies (JSON FRR)
get_ospf_neighbors	ip=172.20.20.12	Voisinage OSPF (JSON FRR)
ping_host	ip=172.20.20.13	4 paquets reçus, 0 % de perte
traceroute_host	ip=172.20.20.13	Chemin réseau en 1 saut
netconf_get	host=172.20.20.11, user, pass	Erreur documentée : port 830 fermé (attendu)

Tableau 19 — Appels d'outils à rejouer dans l'onglet Tools (serveur 2).

4. Chaîne d'analyse complète (serveur 3, port 6474) — scénario D :

Étape	Action graphique	Résultat attendu
1	correlate_alarms (aucun argument)	Un identifiant d'incident, 16 événements groupés sur r1 et r2
2	analyze_root_cause (incident_id)	Les preuves rassemblées pour cet incident
3	generate_incident_report (incident_id + root_cause_summary = « link down sur le lien primaire r1<->r2 »)	Rapport associant le runbook RB-LINK-DOWN, statut EN_ATTENTE_DE_VALIDATIO

		N_HUMAINE
4	detect_anomalies (aucun argument)	4 anomalies et 5 alarmes

Tableau 20 — Chaîne d'analyse de bout en bout dans l'IHM du serveur 3.

D.7 Phase 4 — Niveau 4 : robustesse et injection de panne (4 min, terminal, sudo)

C'est le seul test qui modifie volontairement l'état du réseau. Il coupe le lien primaire r1↔r2, observe la bascule OSPF sur le lien de secours r1↔r3, puis rétablit tout. Aucune configuration n'est écrite de façon permanente.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up
sudo ./lab/fault-injection/inject_bgp_flap.sh status
```

Commande	**Ce que fait la commande**	**Résultat attendu**
inject_bgp_flap.sh status	Affiche l'état nominal sans rien modifier : voisins OSPF de r1, sessions BGP de r1 et route vers le LAN serveur 192.168.3.0/24.	2 voisins Full, 2 sessions iBGP (4 et 5 préfixes reçus)
inject_bgp_flap.sh link_down	Met physiquement hors service le lien primaire r1↔r2 (coupure de l'interface eth1 côté r1). Les protocoles de routage doivent réagir seuls.	Commande exécutée sans erreur
inject_bgp_flap.sh status (pendant la panne)	Re-sonde r1 pendant la coupure pour constater la dégradation et la bascule.	1 voisin OSPF, route 192.168.3.0/24 via le lien de secours 10.0.13.0/30
inject_bgp_flap.sh link_up	Rétablit le lien primaire.	Commande exécutée sans erreur
inject_bgp_flap.sh status (après guérison)	Vérifie le retour à l'état nominal : la redondance a fonctionné dans les deux sens.	2 voisins Full, 2 sessions iBGP (4 et 5 préfixes reçus)

Tableau 21 — Niveau 4 : injection de panne et reconvergence.

Contrôle croisé obligatoire, à faire pendant la panne :

```
# dans l'interface graphique du serveur 2 (port 6374), onglet Tools :
#   snmp_get  ip=172.20.20.11  oid=1.3.6.1.2.1.1.1.0
```

- Résultat attendu : l'appel répond malgré la panne, en signalant l'état réel. C'est la preuve que la supervision ne s'arrête jamais, même en phase dégradée.
- Aucun collecteur ne doit planter pendant la phase dégradée : c'est le critère 9 de la grille.

D.8 Phase 5 — Niveau 5 : sécurité (5 min, terminal)

```
cd /home/floden/mcp-noc-telecom/server
CONSENT_MODE=dry_run ./venv/bin/python demo_securite.py
./venv/bin/pytest test_a6_security.py -q


cd /home/floden/noc-mcp-server && git status --short
grep -R "AuthPass2026" --exclude-dir=.git . 2>/dev/null | head


cd /home/floden/mcp-noc-telecom/server
python3 -c "import json;print([json.loads(l)['status'] for l in
open('audit.log')][-6:])"
```

Commande	**Ce que fait la commande**	**Résultat attendu**
CONSENT_MODE=dry_run … demo_securite.py	Déroule les 5 démonstrations de la couche sécurité : refus d'accès pour rôle insuffisant, autorisation selon le rôle, syslog normal, syslog piégé (injection neutralisée) et impression du journal d'audit.	DEMO 1 à 5 affichées, « FIN DE LA DEMONSTRATION », code retour 0
./venv/bin/pytest test_a6_security.py -q	Relance la suite complète d'acceptation A.6 : RBAC, JWKS, consentement, anti-passthrough, HTTP 401/200, parcours PKCE de bout en bout.	30 passed
git status --short	Affiche l'état du dépôt : vérifie que le fichier .env qui contient les secrets n'est pas suivi par git.	Aucune ligne mentionnant .env
grep -R "AuthPass2026" …	Cherche un mot de passe réel du laboratoire dans tout le dépôt, hors historique git, pour s'assurer qu'aucun secret n'est versionné.	Occurrence uniquement dans .env
python3 -c "… open('audit.log') …"	Lit les 6 dernières entrées du journal d'audit et en extrait le champ status, pour vérifier qu'aucun jeton n'y figure.	Statuts du type AUTORISE, CONSENT_DRY_RUN, REFUSE, ALERTE_INJECTION_NEUTRALIS

		EE

Tableau 22 — Niveau 5 : sécurité, cloisonnement des secrets et audit.

D.9 Phase 6 — Contrôle global de fin de campagne (1 min, terminal)

```
for port in 6274 6374 6474; do \
  curl -s -o /dev/null -w "port $port : HTTP %{http_code}\n"
http://127.0.0.1:$port/; done
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab
```

Commande	Ce que fait la commande	Résultat attendu
curl … port 6274 / 6374 / 6474	Vérifie qu'aucune des trois interfaces MCP Inspector n'est tombée pendant la campagne.	3 × HTTP 200
docker ps … | grep -c mcp-noc-lab	Vérifie que les 5 conteneurs du laboratoire sont toujours actifs en fin de test.	5

Tableau 23 — Contrôle global de fin de campagne.

D.10 Grille de résultats à remplir

À remplir au fil de l'eau : la colonne « Observé » reçoit la valeur réellement obtenue. Cette grille devient directement le tableau de synthèse du support PowerPoint.

N°	Ce qui est vérifié	Commande / action	Attendu	Observé
1	Le laboratoire est déployé	Phase 0 (docker ps)	5 nœuds UP	
2	Le routage converge	Phase 0 (vtysh OSPF / BGP)	Full + 4 et 5 préfixes reçus	
3	La suite unitaire est verte	pytest collector/tests -q	37 passed	
4	Les collecteurs répondent en réel	lab/ validate_collectors.py	13/13	
5	Le serveur expose ≥ 8 outils	Inspector, onglet Tools	9 outils (s.2), 10 outils (s.3)	
6	Resources et prompts servis	Inspector, onglets Resources / Prompts	4 res. + 3 prompts (s.2)	
7	SNMPv3 authPriv fonctionne	Inspector : snmp_get sur 172.20.20.11	success	
8	La collecte CLI fonctionne	Inspector : get_bgp_peers	JSON FRR	
9	La panne est absorbée	Phase 4 (inject_bgp_flap.sh)	Bascule puis guérison	
10	La chaîne d'analyse	Phase 3 bis, serveur 3	RB-LINK-DOWN +	

	aboutit		validation humaine	
11	L'accès non autorisé est refusé	test_mcp.py / demo_securite.py	Refus RBAC	
12	L'injection est neutralisée	demo_securite.py (scénario piégé)	Neutralisation	
13	Aucun secret n'est versionné	git status + grep	.env ignoré	
14	Aucun outil ne modifie le réseau	Revue des outils dans l'IHM	Lecture seule	
15	Authentification OIDC réelle	pytest test_a6_security.py	Jeton accepté, signature falsifiée refusée	
16	OAuth 2.1 + PKCE de bout en bout	pytest test_a6_security.py (flux PKCE)	200 + S256 vérifié	
17	Consentement et anti-passthrough	pytest test_a6_security.py	30 tests verts	

Tableau 24 — Grille de résultats de la campagne (17 contrôles, colonne « Observé » à compléter).

D.11 Correspondance test → exigence → support de présentation

Réf.	Exigence	Test de preuve	Interface
B.2	Faisabilité en laboratoire virtuel	Phase 0	Terminal
B.4	Stack technique ciblée	Phases 1 et 2	Terminal
B.5	Minimum 8 outils MCP	Phase 3 et 3 bis	Terminal + navigateur
B.6	Lecture seule et secrets rigoureux	Phases 3 et 5	Terminal + navigateur
B.7	Analyse sans spéculation, runbook	Phase 3 bis (serveur 3)	Navigateur
B.8	Démo live incident → runbook → validation	Phases 3 bis et 4	Navigateur + terminal
A.2	Outils, resources, prompts	Phase 3 et 3 bis	Terminal + navigateur
A.3	Transports stdio / Streamable HTTP	Phase 5 (tests HTTP)	Terminal
A.4	FastMCP 3.x et décorateur @mcp.tool()	Phase 1	Terminal
A.5	Démonstration via MCP Inspector	Phase 3 bis	Navigateur
A.6	RBAC, OIDC, OAuth 2.1 +	Phases 1, 5 (et parcours	Terminal +

	PKCE, audit	Keycloak)	navigateur

Tableau 25 — Correspondance entre les tests de l'annexe D et les exigences du cahier des charges.

D.12 Points d'attention et écarts constatés

- Un redémarrage de la machine remet l'environnement dans un état partiel : les conteneurs redémarrent mais les commandes d'initialisation de containerlab (sshd, snmpd) ne se rejouent pas. Symptôme : la phase 2 tombe à 3/10 alors que la phase 0 reste verte. Correctif : sudo containerlab deploy -t lab/containerlab/topology.clab.yml (ou redeploy).
- Keycloak ne redémarre pas automatiquement (politique de redémarrage « none »). Symptôme : 10 tests sur 28 sont ignorés (skipped) et test_mcp.py s'arrête sur une erreur de connexion. Correctif : cd /home/floden/mcp-noc-telecom && docker compose up -d.
- Les interfaces MCP Inspector sont des processus, pas des services : elles doivent être relancées manuellement. Correctif : lancer chaque interface avec setsid en spécifiant un port client, un port sandbox et un port d'origine d'application distincts (6274/6275/6279, 6374/6375/6379, 6474/6476/6479).
- Le lanceur MCP Inspector est désormais installé dans le répertoire utilisateur (/home/floden/.npm/_npx/5a9d879542beca3a/...) : la variable INSP doit pointer vers ce chemin, celui de /root n'étant pas accessible sans sudo. Les interfaces doivent être lancées en compte utilisateur, jamais avec sudo, sous peine de recréer des fichiers propriétaires root.
- Le dépôt mcp-noc-ai ne contient pas de test_client_roots.py : le contrôle du serveur 3 se fait exclusivement dans l'IHM (phase 3 bis).
- Chaque redémarrage des interfaces génère de nouveaux jetons d'accès : il faut réextraire les URL avec la commande grep de la phase 3 bis.
- Les chemins des commandes sont relatifs au dépôt indiqué en tête de chaque phase : le cd est obligatoire. Exécuter une phase depuis un autre répertoire renvoie l'erreur « no such file or directory ».
- sudo est réservé à containerlab et docker. Lancer python, pip ou pytest avec sudo rend les fichiers du dépôt propriétaires de root et provoque PermissionError aux exécutions suivantes en compte utilisateur ; le correctif est sudo chown -R <utilisateur> sur les dépôts concernés.
- Le graphe containerlab est un processus séparé : si l'étape 2 indique que le port 50080 est occupé, le graphe tourne déjà et il suffit d'ouvrir http://127.0.0.1:50080 ; sinon il faut le lancer avec la commande de l'étape 2.

Fin du document.